Problem Chosen	University Physics Competition	Team Control Number
B		**609**

Estimation of Firing Velocity and Angle for Artillery

Summary

This paper addresses the historical artillery targeting problem for spherical projectiles fired from cannons 150-200 years ago. To solve this complex ballistic problem, We developed a comprehensive physical model that incorporates several key factors: Air density variation with altitude; Wind effects on trajectory; Drag forces acting on spherical projectiles; Numerical solution of differential equations using the Runge-Kutta method.

Based on our model, we further developed two practical methods suitable for artillery officers trained in mathematics but lacking computational devices: Table method and Approximate Equation Method. The Table Method involves pre-calculating and organizing essential ballistic data into comprehensive reference tables. Artillery officers can quickly consult these tables to determine the required muzzle velocity and firing angle for given target distance, altitude, and wind conditions. The tables provide immediate solutions without requiring complex calculations in the field. While the Approximate Equation Method employs simplified linear equations derived through regression analysis of the comprehensive ballistic data. Key parameters - including air density, muzzle velocity, and firing angle - are represented as simple combinations of basic functions of the input conditions. This approach enables artillery officers to perform rapid manual calculations using straightforward formulas, providing sufficiently accurate solutions for practical field applications.

Both methods successfully bridge the gap between complex physical modeling and practical field requirements, offering historically appropriate solutions that balance mathematical rigor with operational feasibility. The Table Method provides higher accuracy through direct reference, while the Approximate Equation Method offers greater flexibility through simple calculations. Together, they represent a comprehensive solution to the historical artillery targeting problem, demonstrating how mathematical modeling can be adapted for practical military applications in pre-computer eras.

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Restatement of the Problem . . . 3

2 Assumptions **4**

3 Notations **5**

4 Model Force analysis **5**
- 4.1 Gravitational force . . . 5
- 4.2 Air drag force . . . 6
  - 4.2.1 The Variation of Air Density . . . 6
  - 4.2.2 The Variation of C_d . . . 9

5 Model Motion Analysis **11**
- 5.1 Establish the Equations of Motion . . . 11
- 5.2 Control Equations and Boundary Conditions . . . 12
- 5.3 Combined Model . . . 12
- 5.4 Primary Result . . . 14

6 Table Method **16**
- 6.1 Layer 1: Density Calculation . . . 16
- 6.2 Layer 2: Velocity and Angle Calculation . . . 16

7 Approximate Equation Method **17**
- 7.1 Layer 1: Density equation . . . 17
- 7.2 Layer 2: Velocity and Angle equation . . . 18
  - 7.2.1 Fitting method . . . 18
  - 7.2.2 Fitting result analysis . . . 19
  - 7.2.3 Equation and Reference for the artillery officer . . . 21

8 Model Evaluation **21**
- 8.1 Overview . . . 21
- 8.2 Comparison between two methods . . . 21
- 8.3 Combination of the two methods . . . 21
- 8.4 Strength and Weakness . . . 22
  - 8.4.1 Strength . . . 22
  - 8.4.2 Weakness . . . 22
- 8.5 Discussion of Sources of Error . . . 22
  - 8.5.1 Simplified Model Omitting Physical Forces . . . 22
  - 8.5.2 Inherent Numerical Computation Error . . . 23
  - 8.5.3 Limited Fidelity of Empirical Approximations . . . 23
  - 8.5.4 Uncertainties in Practical Operation and Measurement . . . 23

9 Further Discussion **24**
- 9.1 Drag Coefficient Variation . . . 24
- 9.2 Lift and Stability . . . 24

9.3 Ballistic Coefficient 24
10 Report on the Use of AI **25**
Reference **28**
Appendices **29**

1 Introduction

1.1 Background

Hundreds of years ago, cannons were classically deployed to attack the enemies. A classic type of cannon used 150 years ago is the de Bange 90 mm cannon employed by the French army in the late 19th century, as illustrated in Figure 1. However, given the limited measurement tools available at the time, it was impossible to obtain complete environmental data and the accuracy of the instruments was also low. Therefore, an optimal strategy is required for the artillery to estimate the muzzle velocity and firing angle based his knowledge in mathematics and practical experience.



Figure 1: De Bange 90 mm cannon [1]

1.2 Restatement of the Problem

Overall, our task is to establish a model to help the artillery determine the muzzle velocity and firing angle which includes the pitch angle and the yaw angle. For the first part of the problem, the horizontal distance is fixed to 1200 m and the altitudes of the cannon and the target are both fixed to 500 m without the need to consider the wind speed. Then, for the second part of the problem, we are expected to extend our model to fit situations with different horizontal distances, different altitude and different wind speeds. In addition, the model should be accessible enough so that the artillery can use the model without the reliance on computers or calculators.

2 Assumptions

To simplify the problem and make it convenient for us to simulate real-life conditions, we make the following basic assumptions, each of which is properly justified.

- **Target at the same altitude as cannon**. As the problem hasn't clarified the altitude of the target at the second part of the problem, we assume that the target has the same altitude as the cannon.
- **Constant Wind Speed**. The wind speed is assumed to be constant and uniform at all points along the trajectory.
- **A Smooth Spherical Surface**. The surface of the projectile is completely smooth. It would not undergo deformation when launched.
- **Non-Rotating Sphere**. The spherical projectile experiences no rotation or spin effects as the sphere is totally smooth.
- **Constant Gravitational Acceleration**. The gravitational acceleration g is assumed to be constant throughout the trajectory.
- **Initial Barrel Alignment**. The cannon barrel is initially aligned directly towards the target before the artillery adjusts the cannon. Hence, the yaw angle we obtain is based on the condition that the direction of the cannon which is straight towards the target is zero.
- **Negligible Buoyancy Force**. By using $\mathbf{F} = \rho g V$, the buoyancy force for the projectile is about $0.11N$, which is quite small compared with other forces like the air drag, so it is negligible.
- **Negligible Coriolis Force**. By using $\mathbf{F_c} = -2m(\omega \times \mathbf{v})$, the mass of the projectile is $5kg$, the angular velocity of the rotating reference frame (the Earth) is around $7.292 \times 10^{-5} rad/s$, the velocity of the projectile is $0 - 450m/s$,

  $$F_c \leq 2m|\omega||\mathbf{v}| \leq 2 \times 5kg \times (7.292 \times 10^{-5} rad/s) \times 450m/s \approx 0.328N$$

  which is also quite small compared with other forces, so it is negligible.
- **Negligible Vertical Wind Component**. Wind velocity components perpendicular to the ground (vertical wind) are neglected in the analysis.
- **Simplified Weather Conditions**. Neglect the force rain or snow imposes on the projectile when the weather is rainy or snowy. Only the temperature, humidity and air pressure brought by these special weather conditions are considered in the following discussion.

3 Notations

Table 1: Physical symbols used in the ballistic trajectory model

Symbol	Unit	Description
t	s	Time
m	kg	Mass of the spherical projectile
d	m	Diameter of the spherical projectile
H	m	Altitude
D	m	Distance to the target
x	m	Horizontal position coordinate
y	m	Vertical position coordinate
S	m^2	Cross-sectional area of the projectile
v	m/s	Velocity
g	m/s^2	Gravitational acceleration (constant)
ρ	kg/m^3	Air density
θ_1	deg	Launch elevation angle (pitch)
θ_2	deg	Launch azimuth angle (yaw)
F	N	Force
T	$^\circ C$	Temperature
P	Pa	Atmospheric pressure
R	J/ $(kg \cdot K)$	Specific gas constant for air
γ	$^\circ C/km$	Lapse rate of temperature
R_H	–	Relative humidity
C_d	–	Air drag coefficient of the spherical projectile

where we define the main parameters while specific value of those parameters will be given later.

4 Model Force analysis

In the modeling part, we first consider the force that the projectile will be impacted on. As the assumptions have mentioned, the sphere is non-rotating, we don't need to consider the Magnus effect. Similarly, we don't need to take buoyancy force and Coriolis force into account either. Therefore, the total force the projectile takes is the gravitational force, the air drag force. Then we apply the Newton's second law to do the motion analysis along the trajectory in the next section.

4.1 Gravitational force

As we assume that the gravitational acceleration is a constant, the gravitational force is a constant as the mass of the projectile is given. Take the gravitational acceleration as $9.80665 m/s^2$ and therefore,

$$F_g = mg = 5kg \times 9.80665 m/s^2 = 49.03325 N$$

.

4.2 Air drag force

The general formula for the air resistance is given by

$$F_d = \frac{1}{2} C_d \rho S \mathbf{v}^2$$

where ρ stands for the air density, S stands for the cross-sectional area of the projectile and $\mathbf{v}$ stands for the velocity difference between the projectile and the wind.

The cross-sectional area of the projectile is easily found as

$$S = \frac{\pi d^2}{4} = \frac{\pi \cdot (0.11m)^2}{4} = 0.0095m^2$$

.

As the drag coefficient C_d and air density ρ are both functions of $\mathbf{v}$, we will discuss the relationship in the later parts.

4.2.1 The Variation of Air Density

First, let's consider the correlation between the air temperature and the altitude. We assume that the trajectory will always stay in the troposphere, that has an altitude of 0-16 km and according to the an international standard atmosphere defined by the International Civil Aviation Organization, the lapse rate of temperature γ is 6.5 $^\circ\mathrm{C}/km$ [2].

So the Temperature Vertical Distribution Formula is:

$$T(h) = T_0 - \gamma \cdot (h - h_0)$$

where T_0 represents the sea level temperature with the unit $^\circ\mathrm{C}$.

Then, considering the moisture of the air, we have formulas as follows:

$$\rho = \rho_d + \rho_v$$

where ρ_d represents the density of dry air and ρ_v represents the density of vapor.

To calculate the density of vapor, we use the definition of the relative humidity and the Saturation Vapor Pressure Formula (here we use Tetens equation [3] to do approximation):

$$R_H = \frac{e_0}{e_s(T)}$$
$$e_s(T) = 610.78 \cdot e^{\frac{17.27 \cdot T}{T + 237.3}}$$

where R_H is the relative humidity between 0% and 100% , and $e_s(T)$ is the saturation vapor pressure.

From the Ideal Gas Law $P = \rho RT$, the hydrostatic equation $dP = -\rho g dh$ and the definition of Lapse rate, we can obtain the vertical pressure distribution formula:

$$T(h) = T(0) - \lambda h$$

$$dP = -(\frac{P}{RT(h)})gdh$$

$$\frac{dP}{P} = -\frac{-gdh}{R(T(0) - \lambda h)} \rightarrow ln(\frac{P(h)}{P_0}) = \frac{g}{\lambda R} ln(\frac{T(h) + 273.15}{T_0 + 273.15})$$

$$P(h) = P_0 \cdot (\frac{T(h) + 273.15}{T_0 + 273.15})^{\frac{g}{R_d \cdot \gamma}}$$

Furthermore, based on the definition of mixing ratio and Dalton's law of partial pressures, we can obtain the following formulas:

$$P = P_{vapor} + P_{dry}$$

$$w_0 = \frac{m_{vapor}}{m_{dry}} = \frac{\rho_{vapor}}{\rho_{dry}} = \frac{\frac{P_{vapor}}{R_v T}}{\frac{P - P_{vapor}}{R_d T}} = \varepsilon \cdot \frac{P_{vapor}}{P - P_{vapor}} (\varepsilon = \frac{R_d}{R_v} \approx 0.622)$$

$$P_{vapor}(h) = \frac{w_0 \cdot P(h)}{\varepsilon + w_0}$$

$$P_{dry}(h) = P(h) - P_{vapor}(h)$$

where w_0 represents the mixing ratio.

In all, from the Ideal Gas Law, the density of dry air and vapor can be calculated:

$$\rho_d = \frac{P_{dry}(h)}{R_d \cdot (T(h) + 273.15)}$$

$$\rho_v = \frac{P_{vapor}(h)}{R_v \cdot (T(h) + 273.15)}$$

where R_d represents the gas constant for dry air and R_v represents the gas constant for water vapor.

Based on the equations above, we can obtain the corresponding graph for air density with respect to different altitude, different temperature and different humidity and different air pressure that the artillery may obtain in Figure 2, 3, 4.

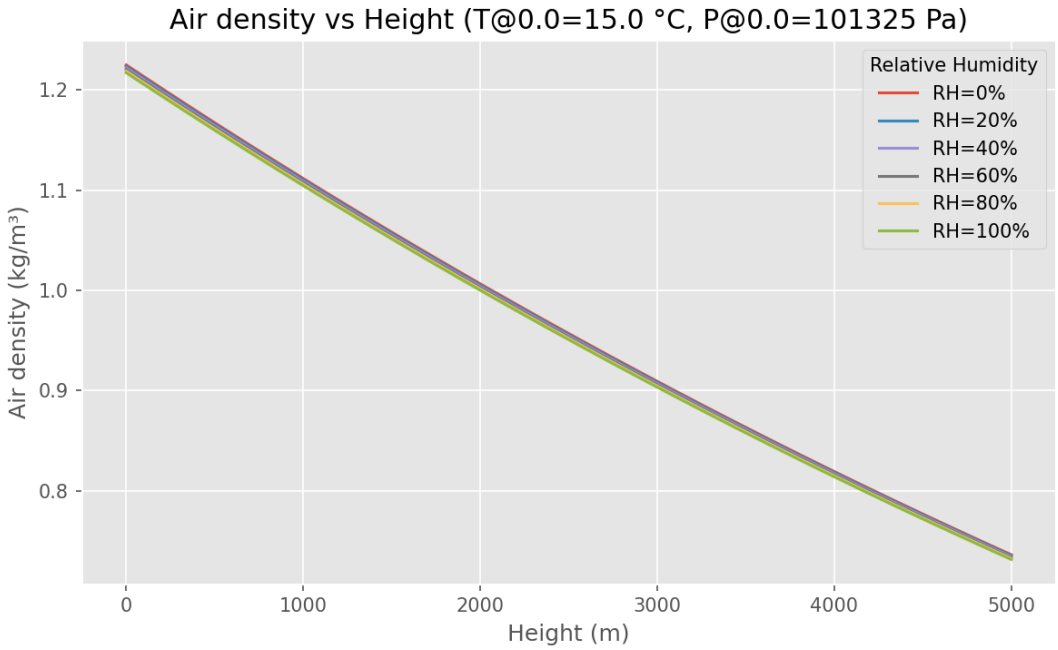


Figure 2: Relationship between air density and altitude

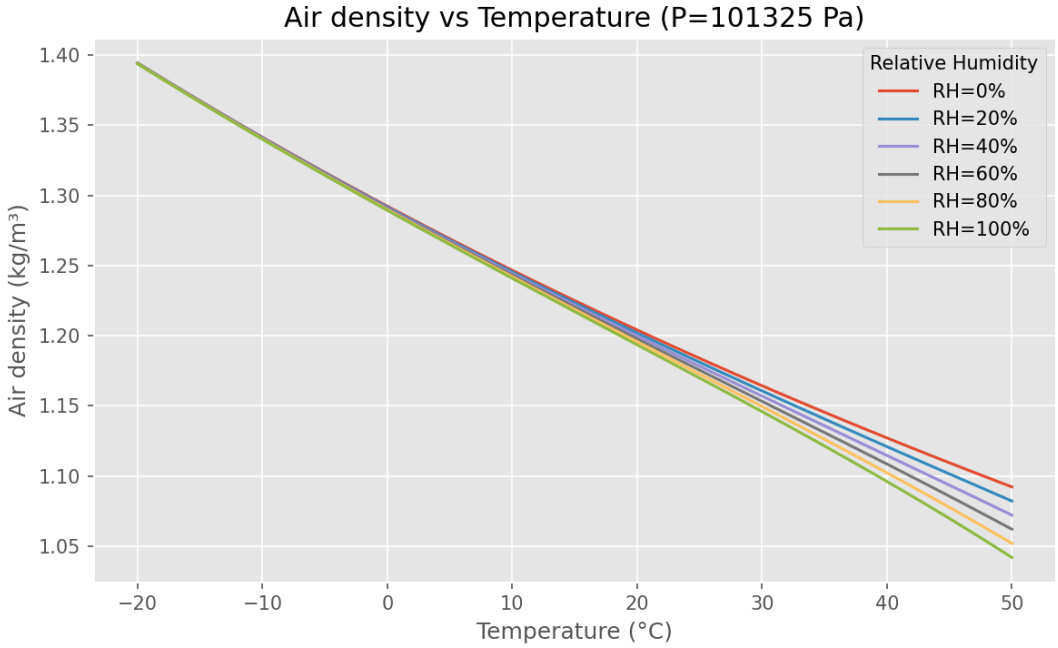


Figure 3: Relationship between air density and temperature

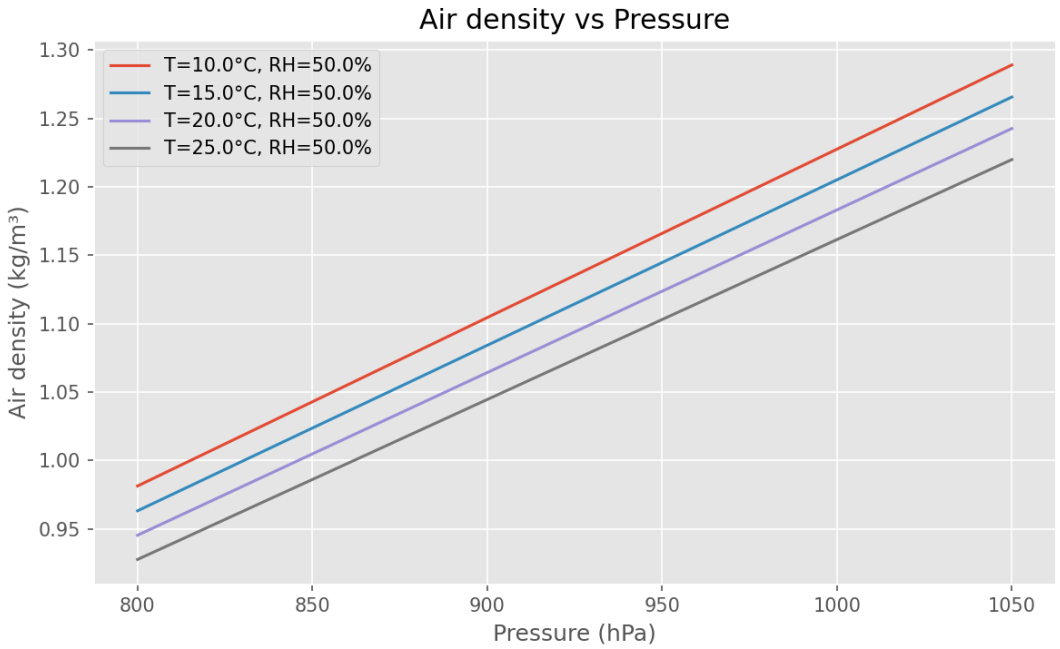


Figure 4: Relationship between air density and air pressure

4.2.2 The Variation of C_d

As velocity changes, Reynolds number ranging between 10^5 to 10^6 changes greatly. So we need to discuss the variation of C_d correspondingly.

To estimate Reynolds number, we can use the following formula:

$$R_e = \frac{\rho v d}{\mu}$$

where ρ is the density of air, v is the relative velocity of the sphere and air, d is the diameter of tube which thus can be approximately seen as the diameter of sphere, and μ is the dynamic viscosity, which is 1.8×10^{-5} roughly for air [4].

For simplicity, we use different formulas for different velocity ranges, due to the nonlinear relationship:

For velocity lower than 340 m/s:

- Range(0, 10) : $C_d = 24/R_e$
- Range(10, 100) : $C_d = 10/\sqrt{R_e}$
- Range(100 , 150): $C_d = 0.9$
- Range(150 , 340): $C_d = 0.47$

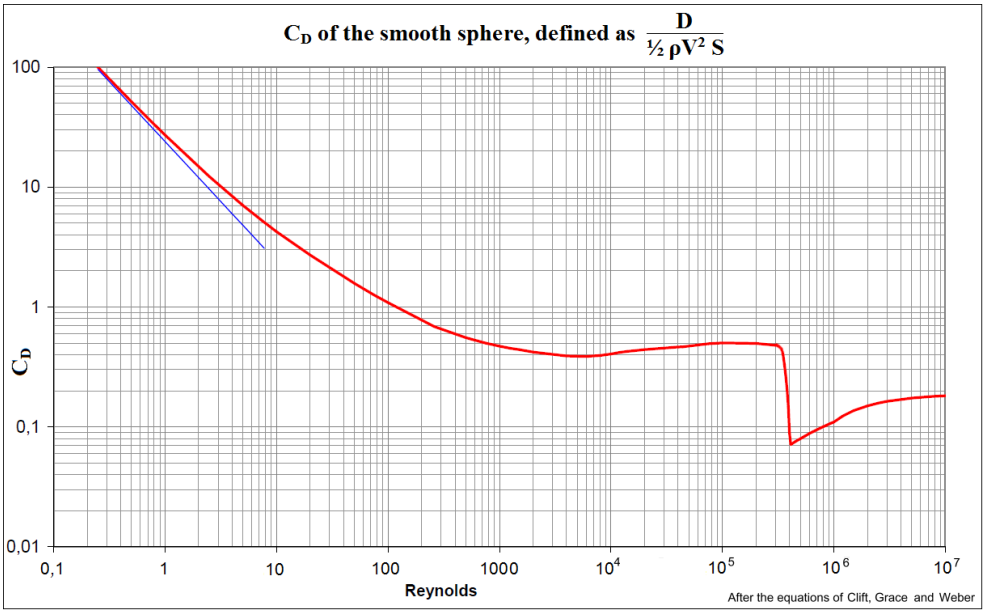


Figure 5: C_D of smooth sphere [5]

For velocity larger than 340 m/s:

$$C_d(Ma) = C_d \times (1 + k \times Ma^n)$$

where k and n is measured in experiment and Ma ranges from 0.8 to 1.2.

As a result, we take the following approximation:

- Mach(1.00 - 1.20): $C_d = 1.2$
- Mach(1.20 - 2.00 transonic spike): $C_d = 0.9$

If more precise calculation is required, some empirical equations can be implemented to calculate C_d [6].

The effect of different C_d when the velocity is high, as shown in Figure 6.

As we restrict the angle from 10 to 25 degree for simulation of actual cannon and there is strong penalty of velocity for supersonic cases, the velocity finally lies mainly in 150 - 200 m/s, which is the most stable C_d zone for calculation, and may potentially perform better in actual use.

Based on the situations given above, we can estimate the order of magnitude of the air drag force:

$$F_d = \frac{1}{2} C_d \rho S \mathbf{v}^2 = \frac{1}{2} \times 0.47 \times 1.2 kg/m^3 \times 0.0095 m^2 \times (200 m/s)^2 = 107.16 N \gg 0.328 N$$

This verifies that the buoyancy force and Coriolis force can be neglected.

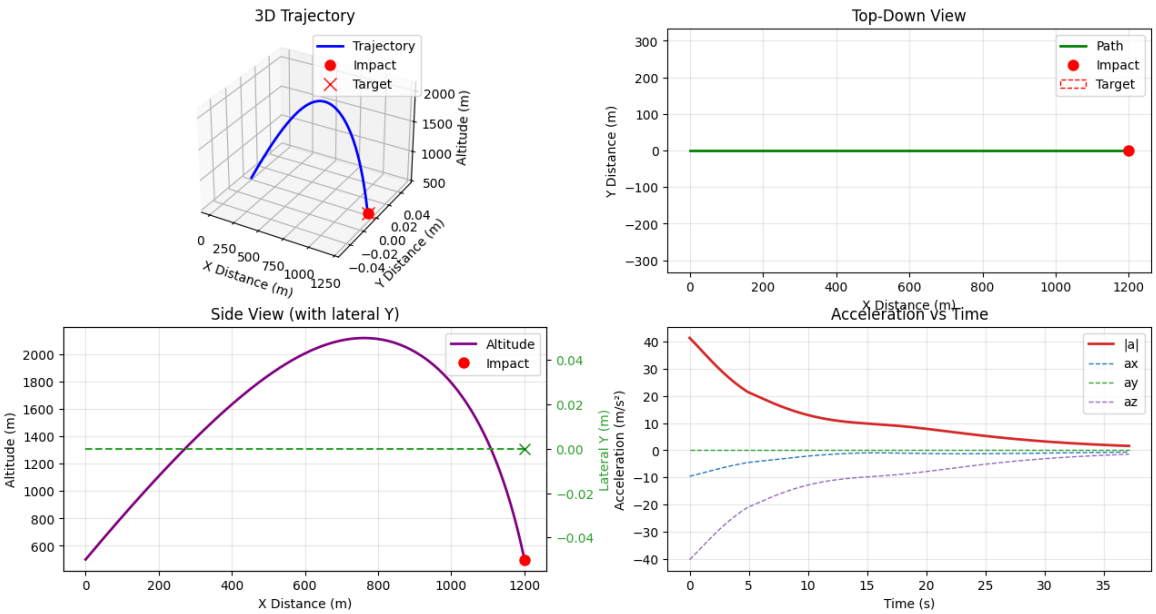


Figure 6: Demonstration of the variation of C_d

5 Model Motion Analysis

During the trajectory of the projectile, it is primarily subjected to two forces: gravity and air resistance. The direction of gravity is vertically downward with a magnitude of $49.03325N$, while the direction of air assistance is determined by the direction of projectile's velocity relative to the air (i.e. the wind), with a magnitude determined by the general formula for the air resistance $F_d = \frac{1}{2}C_d\rho S\mathbf{v}^2$.

5.1 Establish the Equations of Motion

Applying Newton's second law, we obtain the following equations:

$$\frac{d\mathbf{v}_{px}}{dt} = \frac{\mathbf{F}_x}{m} = -\mathbf{i}\cdot\frac{S}{2m}\rho(z)C_d(v)\mathbf{v}\cdot\mathbf{v}_x = \frac{d\mathbf{v_p}}{d\mathbf{x}}\mathbf{v}_{px}$$

$$\frac{d\mathbf{v}_{py}}{dt} = \frac{\mathbf{F}_y}{m} = -\mathbf{j}\cdot\frac{S}{2m}\rho(z)C_d(v)\mathbf{v}\cdot\mathbf{v}_y = \frac{d\mathbf{v_p}}{d\mathbf{y}}\mathbf{v}_{py}$$

$$\frac{d\mathbf{v}_{pz}}{dt} = \frac{\mathbf{F}_z}{m} = -\mathbf{g}-\mathbf{k}\cdot\frac{S}{2m}\rho(z)C_d(v)\mathbf{v}\cdot\mathbf{v}_z = \frac{d\mathbf{v_p}}{d\mathbf{z}}\mathbf{v}_{pz}$$

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}_{px}$$

$$\frac{d\mathbf{y}}{dt} = \mathbf{v}_{py}$$

$$\frac{d\mathbf{z}}{dt} = \mathbf{v}_{pz}$$

$$\mathbf{v} = \mathbf{v}_p - \mathbf{v}_w$$

$$\mathbf{v}_w = \mathbf{v}_{wx} + \mathbf{v}_{wy} + \mathbf{v}_{wz}$$

$$\mathbf{v}_p = \mathbf{v}_{px} + \mathbf{v}_{py} + \mathbf{v}_{pz}$$

5.2 Control Equations and Boundary Conditions

$$\int_0^t (\mathbf{v_p} \cdot \mathbf{i})dt = d$$
$$\int_0^t (\mathbf{v_p} \cdot \mathbf{j})dt = 0$$
$$\int_0^t (\mathbf{v_p} \cdot \mathbf{k})dt = 0$$

$$v_{px} = v_p cos(\theta_1)cos(\theta_2)$$
$$v_{py} = v_p cos(\theta_1)sin(\theta_2)$$
$$v_{pz} = v_p sin(\theta_1)$$

If we consider actual cases for cannon 150 to 200 years ago [1], $\theta_1 \in [10, 25]^\circ$.

5.3 Combined Model

Our model finally includes two main layers:

- Density calculation layer: calculate air density based on different altitude, temperature, humidity and pressure. As a result, the effect of different altitude to the dragging force is mainly considered in density.

$$\rho_d = \frac{\varepsilon \cdot P_0 \cdot (\frac{(T_k - \gamma \cdot (h - h_0))}{T_k})^{\frac{g}{R_d \cdot \gamma}}}{(\varepsilon + w_0) \cdot R_d \cdot (T_k - \gamma \cdot (h - h_0))}$$
$$+ \frac{w_0 \cdot P_0 \cdot (\frac{(T_k - \gamma \cdot (h - h_0))}{T_k})^{\frac{g}{R_d \cdot \gamma}}}{(\varepsilon + w_0) \cdot R_v \cdot (T_k - \gamma \cdot (h - h_0))}$$

$$T_k = T_0 + 273.15$$

The change of ρ demonstrate the effect of different altitude (take the air pressure at the sea level when temperature is 15 $^\circ$C and humidity is 50% to be $101325Pa$):

$$\rho = \rho_0 + kh$$
$$\rho_0 = 1.2094 kg/m^3$$
$$k = -9.92 \times 10^{-5} kg/m^4$$

- Velocity and angle calculation layer: calculate muzzle velocity and angle based on fixed density, horizontal distance, wind force and wind direction.

  Our calculation method and assumptions:

- For all the possible solution, we'll choose the one that takes the least time.
- For the calculation of the differential equation, we'll use Runge Kutta method (RK45).

How the RK45 Method Works: A Step-by-Step Algorithm[7]

Given the initial value problem:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

with current point (t_n, y_n) and step size h, the RK45 method proceeds as follows:

Step 1: Calculate the Six Intermediate Slopes

$$\begin{aligned}
k_1 &= h \cdot f(t_n, y_n) \\
k_2 &= h \cdot f\left(t_n + \tfrac{1}{4}h,\ y_n + \tfrac{1}{4}k_1\right) \\
k_3 &= h \cdot f\left(t_n + \tfrac{3}{8}h,\ y_n + \tfrac{3}{32}k_1 + \tfrac{9}{32}k_2\right) \\
k_4 &= h \cdot f\left(t_n + \tfrac{12}{13}h,\ y_n + \tfrac{1932}{2197}k_1 - \tfrac{7200}{2197}k_2 + \tfrac{7296}{2197}k_3\right) \\
k_5 &= h \cdot f\left(t_n + h,\ y_n + \tfrac{439}{216}k_1 - 8k_2 + \tfrac{3680}{513}k_3 - \tfrac{845}{4104}k_4\right) \\
k_6 &= h \cdot f\left(t_n + \tfrac{1}{2}h,\ y_n - \tfrac{8}{27}k_1 + 2k_2 - \tfrac{3544}{2565}k_3 + \tfrac{1859}{4104}k_4 - \tfrac{11}{40}k_5\right)
\end{aligned}$$

Step 2: Compute Two Estimates for y_{n+1}

$$\text{4th-order estimate:} \quad y_{n+1}^{(4)} = y_n + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5$$

$$\text{5th-order estimate:} \quad y_{n+1}^{(5)} = y_n + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6$$

Step 3: Estimate the Local Truncation Error

$$E = \left| y_{n+1}^{(5)} - y_{n+1}^{(4)} \right|$$

Step 4: Adaptive Step-Size Control

Given a tolerance tol and safety factor $\alpha \approx 0.9$:

- **If $E \leq tol$ (Step Accepted):**
  - Update solution: $y_{n+1} = y_{n+1}^{(5)}$
  - Update time: $t_{n+1} = t_n + h$
  - Calculate new step size:

$$h_{\text{new}} = \alpha \cdot h \cdot \left(\frac{tol}{E}\right)^{1/5}$$

- **If $E > tol$ (Step Rejected):**

- Keep y_n unchanged
- Calculate smaller step size:

$$h_{\text{new}} = \alpha \cdot h \cdot \left(\frac{tol}{E}\right)^{1/5}$$

- Recalculate the step with $h = h_{\text{new}}$

Step 5: Iterate

Repeat the process from Step 1 until the desired end time is reached.

Note: The coefficients are chosen to maximize the order of the method while minimizing the number of function evaluations.

5.4 Primary Result

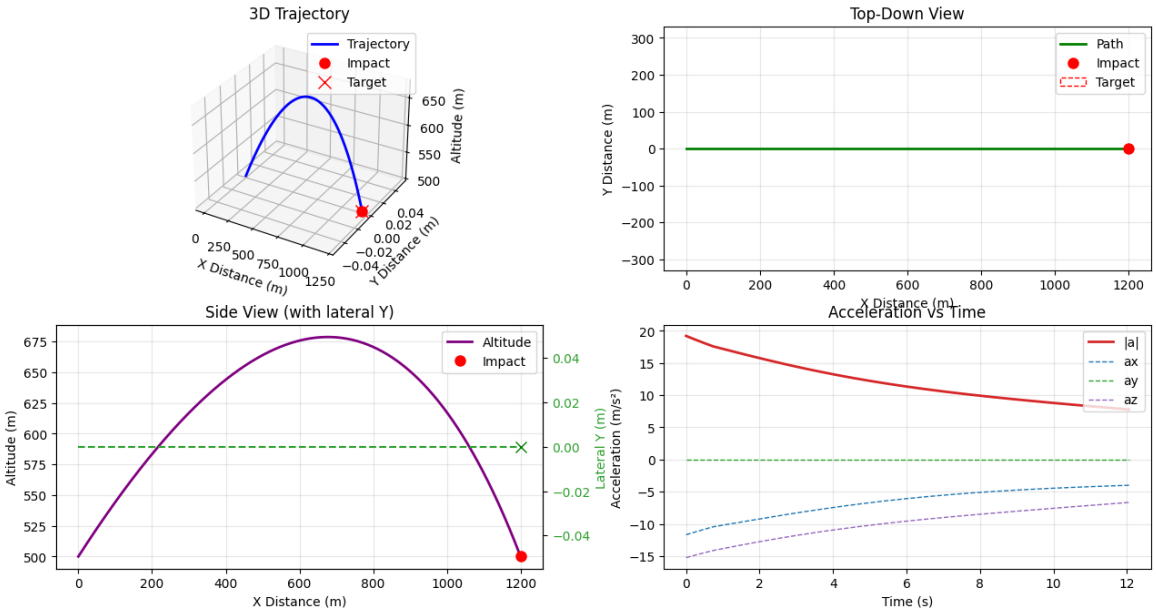


Figure 7: Result of distance 1200 m without wind

$Distance = 1200m, Altitude = 500m$ without wind

- Velocity: $162m/s$
- Degree: $25°$
- Flight Time: $12.05s$

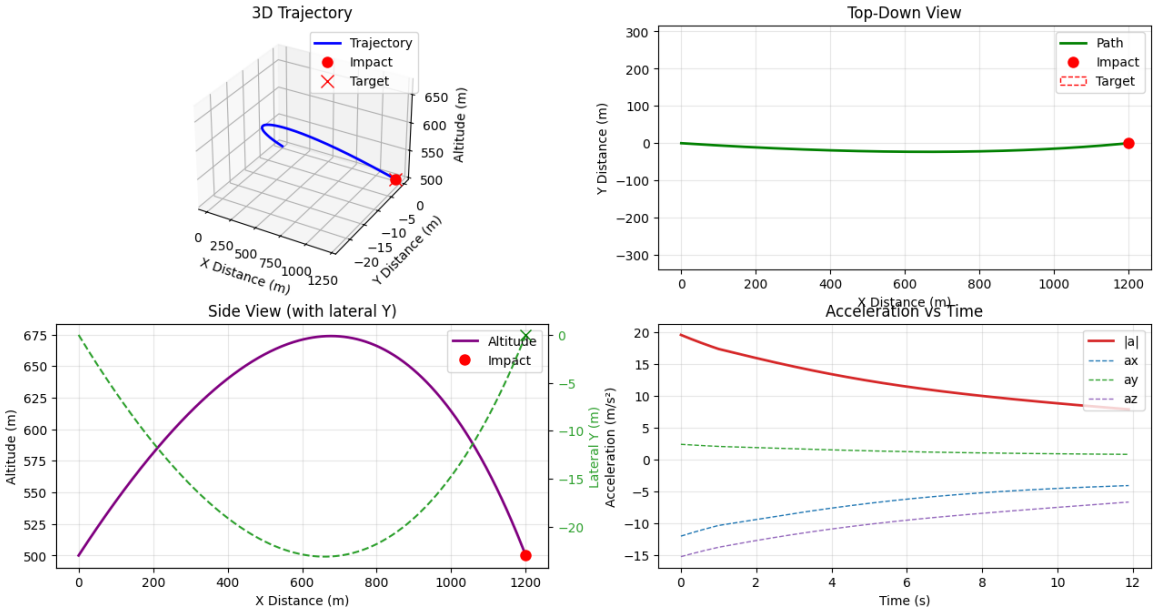


Figure 8: Result of distance 1200 m with strong crosswind for demonstration

$Distance = 1200m, Altitude = 500m$ with $20m/s$ crosswind

- Velocity: $163.7m/s$
- Degree: $24.3°$
- Azimuth Degree: $-3.7°$

Based on this model, we further calculated other initial conditions and developed two practical methods for the artillery officer in real conditions.

6 Table Method

As is stated before, different altitude is considered as a factor of different air density ρ, so we'll just list the table of air density according to different situations.

6.1 Layer 1: Density Calculation

We've made the table of altitude from 0 m to 3000 m in 500 m increments, temperature from 0 °C to 30 °C in 10 °C increments, humidity from 0% to 80 % with 40% increments, air pressure $P = (100000 - 10 \cdot altitude)$ Pa and $(105000 - 10 \cdot altitude)$ Pa. For simple demonstration, we'll just show the table of altitude 500 m. For the artillery to estimate the density of the air around him, he can refer to Table 2 and find the closest data to the observed data or do linear estimation based on the difference between them. The whole table will be shown in the appendix.

T [°C]	R_h[%]	altitude[m]	P[Pa]	ρ[kg/m^3]	P[Pa]	ρ[kg/m^3]
0	0	500	95000	1.1514	100000	1.212
0	40	500	95000	1.1503	100000	1.2109
0	80	500	95000	1.1492	100000	1.2098
10	0	500	95000	1.1128	100000	1.1714
10	40	500	95000	1.1106	100000	1.1692
10	80	500	95000	1.1084	100000	1.167
20	0	500	95000	1.0766	100000	1.1333
20	40	500	95000	1.0726	100000	1.1293
20	80	500	95000	1.0686	100000	1.1253
30	0	500	95000	1.0428	100000	1.0976
30	40	500	95000	1.0357	100000	1.0906
30	80	500	95000	1.0287	100000	1.0836

Table 2: Air density table for altitude 500 m

6.2 Layer 2: Velocity and Angle Calculation

As the density is calculated, we can then continue to calculate the velocity and angle. (Note that various altitude is included in density calculation, and officer can simply calculate density based on only altitude according to the linear equation if he doesn't want to use table.)

We've made the table of density from 1.1 kg/m^3 to 1.3 kg/m^3 with 0.1 kg/m^3 increments, distance from 1000 m to 1500 m with 100 m increments, wind direction from 0 ° to 180 ° with 30 ° increments since it's symmetric. For simple demonstration, we'll just show the table of density 1.2 kg/m^3, distance 1300 m. The whole table will also be shown in the appendix.

The table doesn't take extreme weather conditions into account, as it may cause unexpected results very different from just solving the differential equation and may need to use complex CFD simulation. Wind scale table in the appendix shows that wind speed faster than $10m/s$ is no longer considered as normal weather, and the example table can be easily extend if time for calculation permits.

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1300	0.0	0	177.3	24.0	-0.0	0.0
1300	2.5	0	176.5	23.6	0.0	0.0
1300	5.0	0	172.5	25.0	0.0	0.0
1300	7.5	0	171.1	25.0	0.0	0.0
1300	10.0	0	169.8	25.0	0.0	0.0
1300	0.0	30	177.3	24.0	-0.0	0.0
1300	2.5	30	197.9	16.6	-0.2	0.0
1300	5.0	30	197.7	16.4	-0.3	0.0
1300	7.5	30	171.7	24.9	-0.7	0.0
1300	10.0	30	170.6	24.9	-1.0	0.0
1300	0.0	60	177.3	24.0	-0.0	0.0
1300	2.5	60	197.7	16.8	-0.3	0.0
1300	5.0	60	176.2	23.8	-0.8	0.0
1300	7.5	60	173.6	24.8	-1.3	0.0
1300	10.0	60	197.9	16.3	-1.2	0.0
1300	0.0	90	177.3	24.0	-0.0	0.0
1300	2.5	90	177.2	24.0	-0.5	0.0
1300	5.0	90	177.4	24.0	-1.0	0.0
1300	7.5	90	177.4	24.0	-1.5	0.0
1300	10.0	90	177.9	23.8	-1.9	0.0
1300	0.0	120	177.3	24.0	-0.0	0.0
1300	2.5	120	177.7	24.1	-0.4	0.0
1300	5.0	120	197.3	17.3	-0.6	0.0
1300	7.5	120	198.5	17.2	-1.0	0.0
1300	10.0	120	179.2	24.7	-1.8	0.0
1300	0.0	150	177.3	24.0	-0.0	0.0
1300	2.5	150	178.4	24.1	-0.2	0.0
1300	5.0	150	178.3	24.9	-0.5	0.0
1300	7.5	150	179.8	24.9	-0.8	0.0
1300	10.0	150	182.3	24.2	-1.0	0.0
1300	0.0	180	177.3	24.0	-0.0	0.0
1300	2.5	180	177.4	24.8	0.0	0.0
1300	5.0	180	178.6	25.0	-0.0	0.0
1300	7.5	180	181.6	24.2	-0.0	0.0
1300	10.0	180	199.0	17.9	-0.0	0.0

Table 3: Firing table for air density 1.2 kg/m^3 at distance 1300 m

7 Approximate Equation Method

7.1 Layer 1: Density equation

We use SPSSPRO [8] to help do linear approximation of the air density with respect to temperature, relative humidity, altitude and air pressure based on the table in the appendix. Thus

the following linear equation is obtained

$$\rho = -3.68 \times 10^{-3}T - 9 \times 10^{-5}R_h - 1.05 \times 10^{-4}altitude + 1.12 \times 10^{-5}P + 0.1431$$

with $R^2 = 0.996$, meaning that there exists a strong linear correlation among these variables.

7.2 Layer 2: Velocity and Angle equation

7.2.1 Fitting method

- Huber Regression

  Huber regression is a robust statistical technique that combines the advantages of both least squares (L_2) and least absolute deviation (L_1) regression. It uses a hybrid loss function that is quadratic for small residuals and linear for large residuals, providing robustness to outliers while maintaining efficiency for normal data.

  The Huber loss function is defined as:

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{for } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{for } |r| > \delta \end{cases}$$

  where $r = y - \hat{y}$ is the residual and δ is a threshold parameter that controls the transition between quadratic and linear behavior. Typical values for δ range from 1.0 to 2.0.

  The optimization problem minimizes:

$$\min_{\beta} \sum_{i=1}^{n} L_\delta(y_i - X_i\beta)$$

  Key properties:

  - **Robustness**: Less sensitive to outliers than ordinary least squares
  - **Efficiency**: Maintains good statistical properties for normal errors
  - **Continuity**: Loss function is continuously differentiable

- Physical constraint based on the distribution of data

- Adaptive weight strategy Adaptive weighting uses Gaussian emphasis to prioritize samples near operational centers while maintaining full dataset utilization:

$$w_i = \exp\left(-\frac{1}{2}\left(\frac{y_i - \mu}{\sigma}\right)^2\right)$$

  where μ is the target center (typically median or specified peak) and σ controls the focus breadth. Weights are normalized:

$$\tilde{w}_i = \frac{w_i}{\frac{1}{n}\sum_{j=1}^{n} w_j}$$

This approach provides smooth emphasis transitions and maintains mathematical stability through proper normalization.

- Focused training

  Focused training restricts model fitting to specific operational ranges, prioritizing accuracy in militarily relevant parameter regions. The method employs hard thresholding:

$$\mathcal{D}_{\text{focused}} = \{(x_i, y_i) \in \mathcal{D} \mid y_{\min} \leq y_i \leq y_{\max}\}$$

  where $y_{\min}$ and $y_{\max}$ define the operationally critical range (e.g., elevation angles 23°-25°). This ensures optimal performance where it matters most, while ignoring outliers and irrelevant data regions.

  Key Benefits:

  - **Focused Training**: Maximum accuracy in critical operational ranges
  - **Adaptive Weighting**: Balanced emphasis with full data utilization
  - **Operational Relevance**: Models optimized for real-world usage patterns

7.2.2 Fitting result analysis

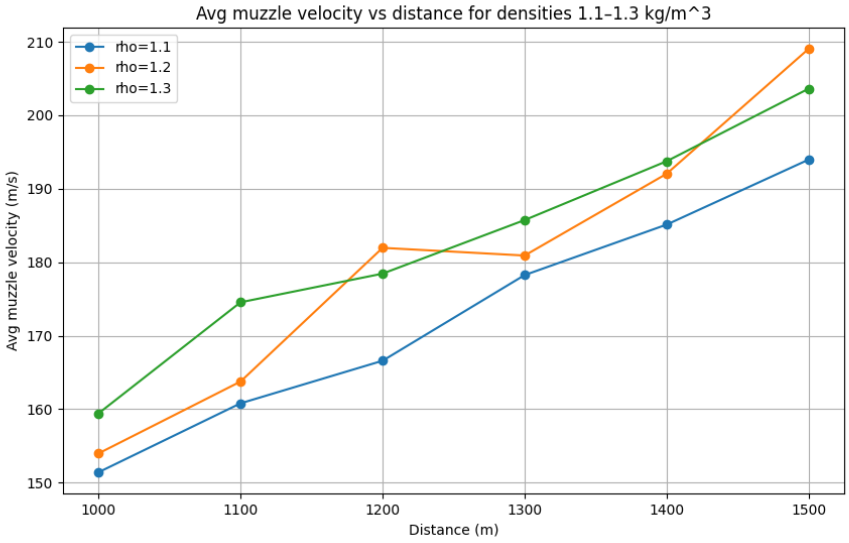
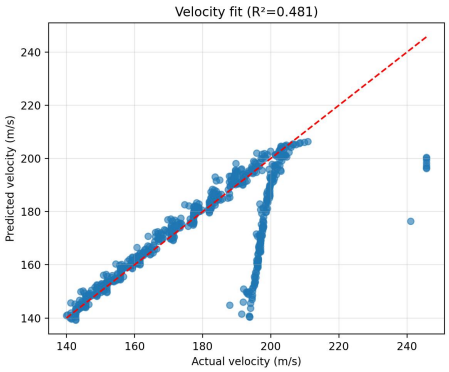
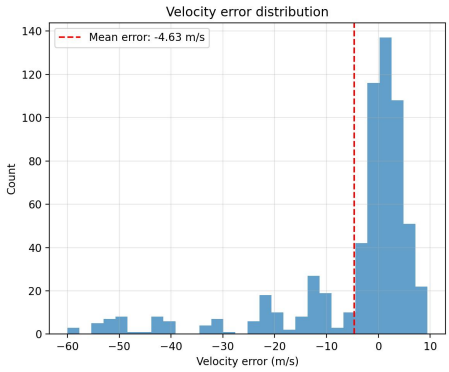


Figure 9: sample average velocity of all densities

- Velocity : two almost linear result with one major distribution
- Azimuth Angle: almost linear
- Elevation Angle: focus near above range 25

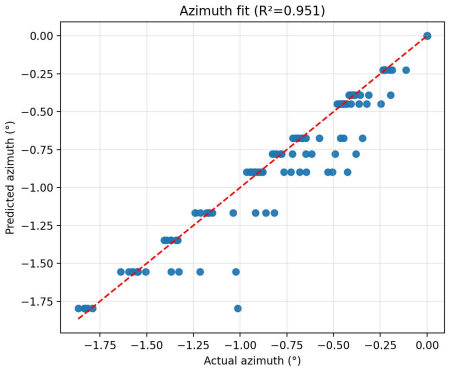


(a) velocity scatter

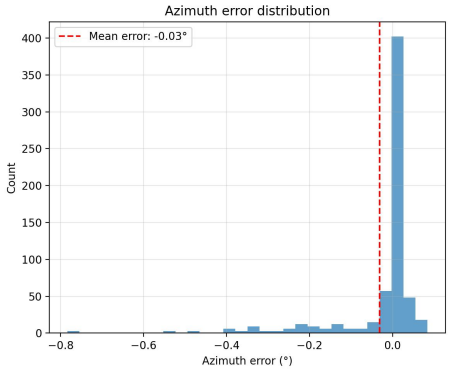


(b) velocity histogram

Figure 10: velocity fit

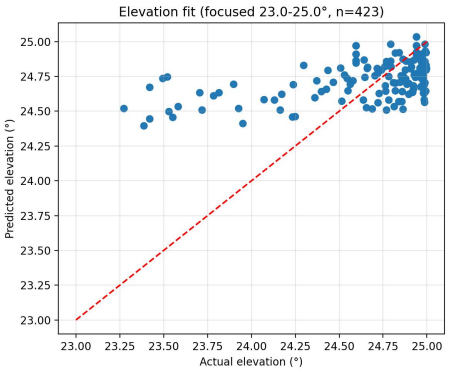


(a) azimuth scatter

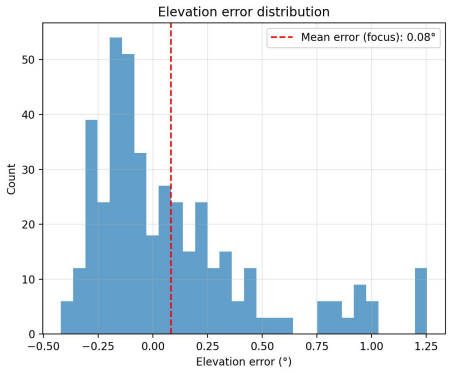


(b) azimuth histogram

Figure 11: azimuth fit



(a) elevation scatter



(b) elevation histogram

Figure 12: elevation fit

7.2.3 Equation and Reference for the artillery officer

- Velocity Formula:

$$v = 18.8 + 0.1094 \times D - 0.1548 \times v_w - 2.0527 \times v_w \times cos(\phi) + \frac{17.1294}{\sqrt{\rho}}$$

- Elevation Formula:

$$\theta = 25.6 - 0.0006 \times D - 0.0204 \times v_w - 0.1262 \times v_w \times sin(\phi)$$

- Azimuth Formula:

$$\alpha = -0.1797 \times v_w \times sin(\phi)$$

- Quick reference:
  - Per $+100m$ distance, velocity changes $\approx 10.9m/s$
  - Per $+1m/s$ headwind, velocity changes $\approx -2.207m/s$
  - Density change from 1.2 to 1.1 causes velocity change $\approx 0.7m/s$

  Where:

  - D: target distance (m), typical range: 1000-1500 m
  - v_w: wind speed (m/s)
  - ϕ: wind direction (degrees), convention used in features
  - ρ: air density (kg/m^3), recommended used range: 1.1-1.3

8 Model Evaluation

8.1 Overview

This ballistic model is a complex physical model that comprehensively considers multiple factors including air density variation with altitude, crosswind effects, rainfall influence, etc. It employs the Runge-Kutta method for numerical solution, providing scientific computational methods for projectile trajectory prediction.

8.2 Comparison between two methods

Table method may be more simple to use for the officer, but it need to balance the content length and the effectiveness of table, or the table is too long for searching or there may be many situations that the officer can't find an approximate answer. The approximate equation method may let the officer do more calculation of multiplication. Well-trained as he is, it may not be a major difficulty. However, the equation is not as precise as the table and may cause the projectile to miss the target.

8.3 Combination of the two methods

In reality, the artillery officer can look up the table for a rough range, then he could calculate a more precise velocity and angle by using the "Quick Reference" from the approximation equation method in the last section. This could be much quicker than solely using those two method.

8.4 Strength and Weakness

8.4.1 Strength

- We analyze the problem based on many authoritative models like the ICAO Standard Atmosphere, making our model of great validity.
- Our model is fairly applicable to the artillery owing to our careful consideration of the reality including the cannon parameters and tools 150 or 200 years ago.
- We use Runge-Kutta method to calculate ballistic model, which exhibits excellent numerical stability.
- We use some random data to simulate different situations that the artillery may face in the reality. Then we impose error analysis between our simplified equation given to the artillery and the more accurate and complex model we get. The fitting method has been verified according to the histogram and R^2 value listed above.
- We combine the advantages of both measurements. For the table method, there exists no error while the approximate equation method can help make the estimation more flexible to the input and bring about little error, making the whole model to be robust.

8.4.2 Weakness

- Our model uses many empirical and approximation formulas to simplify the analysis and calculation, which may not be so reasonable in the real life. We do not use CFD to do simulation, and omit extreme cases for simplicity.
- There still exist many sources of errors, which will be discussed in the following chapter.
- The computational complexity is relatively high. It may take some time to complete the whole table for military use.

8.5 Discussion of Sources of Error

There are many reasons that can lead to error, here we list some of them.

8.5.1 Simplified Model Omitting Physical Forces

Our model treats the Earth as an inertial reference frame, neglecting physical forces arising from its rotation and composition. Notably, the Coriolis effect, which induces a slight but persistent lateral deflection of the trajectory, is ignored. Furthermore, minuscule electromagnetic effects from the Earth's magnetic field on the metal projectile are disregarded. While these omissions result in negligible errors for short-range engagements (e.g., 1-2 km), they become sources of systematic bias when pursuing extreme precision or calculating very long-range trajectories. Also the model assumes the projectile is a non-rotating, perfectly smooth sphere. In reality, firing can impart spin due to rifling or asymmetric forces. This rotation generates a lift or lateral force perpendicular to the velocity and spin vectors via the Magnus effect, altering the projectile's path and stability. Ignoring this effect introduces a discrepancy between the model and physical reality, leading to unaccounted-for trajectory curvature.

8.5.2 Inherent Numerical Computation Error

Even with a perfect physical model, solving the differential equations numerically on a computer (e.g., using the Runge-Kutta method) introduces intrinsic errors. These include truncation error (because the numerical algorithm itself is an approximation) and round-off error (due to the finite precision of computer arithmetic). Although we mitigate these by using high-order methods and small step sizes—making them typically much smaller than model errors—they remain an unavoidable, minor gap between the theoretical and computed solution.

8.5.3 Limited Fidelity of Empirical Approximations

To enable rapid estimation, we employed linear or low-order polynomial fits to represent highly non-linear ballistic relationships. This approximation inevitably produces fitting residuals between the simplified curve and the true complex function. The error of these empirical formulas increases significantly when operating outside the central range of the dataset used for fitting, limiting their predictive power and ability to capture all nuances of the ballistic parameters.

8.5.4 Uncertainties in Practical Operation and Measurement

This error source exists outside the pure mathematical model but is critically important. It encompasses random uncertainties from measuring target range and altitude, setting the cannon's elevation and azimuth, variations in individual projectile mass, and fluctuations in propellant charge quantity and quality. These factors introduce unpredictable random errors in each shot, causing the actual impact points to form a dispersion pattern around the intended target. The discrete nature of the tabulation method, which cannot list every possible scenario, further amplifies the uncertainty introduced by the need to estimate and interpolate between table entries.

9 Further Discussion

The physical model developed in this study assumes a spherical projectile, which was common in artillery 150-200 years ago. However, the shape of a projectile significantly influences its trajectory due to aerodynamic effects. In this section, we discuss the potential impacts of projectile shape and how our model could be extended to account for non-spherical projectiles.

9.1 Drag Coefficient Variation

The drag coefficient C_d is highly dependent on the shape of the projectile. While a sphere has relatively large value of C_d in the subsonic regime, streamlined shapes such as an ogive or cone can have smaller value of C_d. This reduction in drag would result in a flatter trajectory and longer range for the same initial velocity.

9.2 Lift and Stability

Non-spherical projectiles may generate lift if they are oriented at an angle of attack. This lift force can alter the trajectory, potentially increasing range but also introducing complexity due to the need for stability. Projectiles are designed to be statically stable, meaning the center of pressure is located behind the center of mass. This is achieved through specific shape designs and, in some cases, by imparting spin (via rifling) to provide gyroscopic stability. The Magnus effect, which is the lateral force on a spinning projectile, can also cause drift and require corrections.

9.3 Ballistic Coefficient

The ballistic coefficient (BC), defined as $BC = \frac{m}{C_d S}$, is a measure of a projectile's ability to overcome air resistance. A higher BC indicates a more efficient projectile that retains velocity better and is less affected by wind. Spherical projectiles have a relatively low BC due to their high C_d and large cross-sectional area for their mass. Streamlined projectiles have higher BC, which is why modern artillery shells are long and slender.

10 Report on the Use of AI

- GPT5-mini

  Input prompt: how to collect the result of the program xxx.py and set output file xxx.csv in table.

  Output:

````
Short answer: make your program produce a tabular

object (list-of-dicts or pandas.DataFrame) and

write it to CSV (or print CSV to stdout and

redirect). Below are minimal, copy-paste examples.

1) Add an --output arg and save a DataFrame (recommended)
‘‘‘‘python
# ...existing code...
import argparse
import pandas as pd

def main():
    parser = argparse.ArgumentParser()

    parser.add_argument('--output', '-o', help='CSV

    output path', default='xxx.csv')

    args = parser.parse_args()

    # ... run your computation, produce ‘rows‘ as

    list of dicts or a DataFrame ...

    # example: rows = [{'col1':1,'col2':2}, ...]

    df = pd.DataFrame(rows)   # or your existing

    DataFrame

    df.to_csv(args.output, index=False)
    print(f"Wrote results to {args.output}")

if __name__ == '__main__':
    main()
````

````
# ...existing code...
````

Usage:
- python c:\25UPC\src\xxx.py --output results.csv

2) If you prefer writing without pandas, use csv.DictWriter

````python
# not tied to a file path example
import csv

rows = [{'a':1,'b':2},{'a':3,'b':4}]
out = 'xxx.csv'
with open(out, 'w', newline='') as fh:
    writer = csv.DictWriter(fh,

    fieldnames=rows[0].keys())

    writer.writeheader()
    writer.writerows(rows)
````

3) Quick stdout CSV + shell redirect

- inside script: print(df.to_csv(index=False)) or

print(','.join(header)); print rows...

- then run: python xxx.py > xxx.csv

Which to use?
- Use pandas.to_csv when you already have a

DataFrame.
- Use csv.DictWriter for light-weight dependency-

free writing.
- Use stdout redirect if you cannot modify the

script but it can print CSV.

If you want, tell me which script (path) to modify

and whether it already produces `rows` or

‘DataFrame‘ and I will return the exact patch.

References

[1] "De Bange 90 mm cannon," *Wikipedia: The Free Encyclopedia*. Wikimedia Foundation, Sept. 18, 2025. [Online]. Available: https://en.wikipedia.org/wiki/De_Bange_90_mm_cannon#cite_ref-1 [Accessed: Nov. 9, 2025]

[2] International Civil Aviation Organization, Manual of the ICAO Standard Atmosphere (Extended to 80 Kilometres (262 500 Feet)), 3rd ed. Montreal, QC, Canada: International Civil Aviation Organization, 1993.

[3] J. L. Monteith and M. H. Unsworth, *Principles of environmental physics: plants, animals, and the atmosphere,* 4th ed. Amsterdam, The Netherlands: AP, 2013. [Online]. Available: https://books.google.ch/books?id=EhLtk6hkLogC&pg=PA13

[4] "Drag of a Sphere," *Glenn Research Center*, June. 30, 2025. Available: https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/drag-of-a-sphere/#experimental-observations-of-reynolds-number [Accessed: Nov. 9, 2025]

[5] "Drag coefficient of a sphere as a function of Reynolds number," *Wikimedia Commons*.

[6] Hongli Yang, Minqiang Fan, Airong Liu, Lianping Dong, General formulas for drag coefficient and settling velocity of sphere based on theoretical law, *International Journal of Mining Science and Technology*, Volume 25, Issue 2, 2015, Pages 219-223, ISSN 2095-2686, https://doi.org/10.1016/j.ijmst.2015.02.009.

[7] J. H. Mathews and K. D. Fink, *Numerical Methods Using MATLAB, 4th ed. Upper Saddle River,* N.J.: Pearson, 2004, pp. 497-499.

[8] "SPSSPRO," Available: https://www.spsspro.com/

Appendices

Tables

Wind speed

Wind Force	Wind Speed (m/s)	Wind Speed (km/h)	Wind Speed (knots)	Name	Land Conditions / Sea Conditions
0	0.0-0.2	<1	<1	Calm	Calm, smoke rises vertically / Sea like a mirror
1	0.3-1.5	1-5	1-3	Light air	Smoke shows wind direction / Small wavelets
2	1.6-3.3	6-11	4-6	Light breeze	Wind felt on face / Small waves, crests glassy
3	3.4-5.4	12-19	7-10	Gentle breeze	Flags extended / Large wavelets, crests begin to break
4	5.5-7.9	20-28	11-16	Moderate breeze	Dust and loose paper raised / Small waves, becoming longer
5	8.0-10.7	29-38	17-21	Fresh breeze	Small trees in leaf begin to sway / Moderate waves, some foam
6	10.8-13.8	39-49	22-27	Strong breeze	Large branches in motion / Large waves, foam crests
7	13.9-17.1	50-61	28-33	Near gale	Whole trees in motion / Sea heaps up, foam from breaking waves
8	17.2-20.7	62-74	34-40	Gale	Twigs break off trees / Moderately high waves, foam blown in streaks
9	20.8-24.4	75-88	41-47	Strong gale	Slight structural damage / High waves, dense foam
10	24.5-28.4	89-102	48-55	Storm	Trees uprooted / Very high waves, sea surface white
11	28.5-32.6	103-117	56-63	Violent storm	Widespread damage / Exceptionally high waves
12	≥32.7	≥118	≥64	Hurricane	Devastation / Huge waves, air filled with foam

Air density table

T [°C]	R_H[%]	height[m]	P[Pa]	ρ[kg/m^3]	P[Pa]	ρ[kg/m^3]
0	0	0	100000	1.2754	105000	1.3392
0	40	0	100000	1.2742	105000	1.338
0	80	0	100000	1.273	105000	1.3368
10	0	0	100000	1.2303	105000	1.2919
10	40	0	100000	1.2281	105000	1.2896
10	80	0	100000	1.2258	105000	1.2873
20	0	0	100000	1.1884	105000	1.2478
20	40	0	100000	1.1842	105000	1.2436
20	80	0	100000	1.18	105000	1.2394
30	0	0	100000	1.1492	105000	1.2066
30	40	0	100000	1.1418	105000	1.1993
30	80	0	100000	1.1344	105000	1.1919
0	0	500	95000	1.1514	100000	1.212
0	40	500	95000	1.1503	100000	1.2109
0	80	500	95000	1.1492	100000	1.2098
10	0	500	95000	1.1128	100000	1.1714
10	40	500	95000	1.1106	100000	1.1692
10	80	500	95000	1.1084	100000	1.167
20	0	500	95000	1.0766	100000	1.1333
20	40	500	95000	1.0726	100000	1.1293
20	80	500	95000	1.0686	100000	1.1253
30	0	500	95000	1.0428	100000	1.0976
30	40	500	95000	1.0357	100000	1.0906
30	80	500	95000	1.0287	100000	1.0836
0	0	1000	90000	1.036	95000	1.0936
0	40	1000	90000	1.035	95000	1.0925
0	80	1000	90000	1.0339	95000	1.0915
10	0	1000	90000	1.0031	95000	1.0588
10	40	1000	90000	1.001	95000	1.0568
10	80	1000	90000	0.999	95000	1.0547
20	0	1000	90000	0.9722	95000	1.0262
20	40	1000	90000	0.9684	95000	1.0224
20	80	1000	90000	0.9646	95000	1.0186
30	0	1000	90000	0.9431	95000	0.9955
30	40	1000	90000	0.9364	95000	0.9888
30	80	1000	90000	0.9297	95000	0.9821
0	0	1500	85000	0.9287	90000	0.9833
0	40	1500	85000	0.9277	90000	0.9823
0	80	1500	85000	0.9267	90000	0.9813
10	0	1500	85000	0.9009	90000	0.9539
10	40	1500	85000	0.8989	90000	0.9519
10	80	1500	85000	0.897	90000	0.95

T [°C]	$R_H[\%]$	height[m]	P[Pa]	ρ[kg/m^3]	P[Pa]	ρ[kg/m^3]
20	0	1500	85000	0.8747	90000	0.9261
20	40	1500	85000	0.871	90000	0.9225
20	80	1500	85000	0.8674	90000	0.9189
30	0	1500	85000	0.8499	90000	0.8999
30	40	1500	85000	0.8435	90000	0.8935
30	80	1500	85000	0.8371	90000	0.8871
0	0	2000	80000	0.8291	85000	0.8809
0	40	2000	80000	0.8281	85000	0.88
0	80	2000	80000	0.8272	85000	0.879
10	0	2000	80000	0.8058	85000	0.8562
10	40	2000	80000	0.804	85000	0.8543
10	80	2000	80000	0.8021	85000	0.8525
20	0	2000	80000	0.7838	85000	0.8328
20	40	2000	80000	0.7803	85000	0.8293
20	80	2000	80000	0.7769	85000	0.8259
30	0	2000	80000	0.7629	85000	0.8106
30	40	2000	80000	0.7568	85000	0.8045
30	80	2000	80000	0.7507	85000	0.7983
0	0	2500	75000	0.7368	80000	0.7859
0	40	2500	75000	0.7359	80000	0.785
0	80	2500	75000	0.735	80000	0.7841
10	0	2500	75000	0.7175	80000	0.7654
10	40	2500	75000	0.7158	80000	0.7636
10	80	2500	75000	0.714	80000	0.7618
20	0	2500	75000	0.6992	80000	0.7458
20	40	2500	75000	0.6959	80000	0.7425
20	80	2500	75000	0.6926	80000	0.7392
30	0	2500	75000	0.6817	80000	0.7272
30	40	2500	75000	0.6759	80000	0.7214
30	80	2500	75000	0.6701	80000	0.7155
0	0	3000	70000	0.6514	75000	0.6979
0	40	3000	70000	0.6505	75000	0.6971
0	80	3000	70000	0.6497	75000	0.6962
10	0	3000	70000	0.6357	75000	0.6811
10	40	3000	70000	0.634	75000	0.6794
10	80	3000	70000	0.6323	75000	0.6777
20	0	3000	70000	0.6206	75000	0.6649
20	40	3000	70000	0.6175	75000	0.6618
20	80	3000	70000	0.6143	75000	0.6587
30	0	3000	70000	0.6062	75000	0.6495
30	40	3000	70000	0.6006	75000	0.6439
30	80	3000	70000	0.5951	75000	0.6384

Artillery Firing Tables

Air Density = 1.1 kg/m^3

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1000	0.0	0	143.1	24.9	0.0	0.0
1000	2.5	0	142.4	24.8	0.0	0.0
1000	5.0	0	193.9	10.7	-0.0	0.0
1000	7.5	0	193.8	10.6	0.0	0.0
1000	10.0	0	140.3	24.3	-0.0	0.0
1000	0.0	30	143.1	24.9	0.0	0.0
1000	2.5	30	142.5	24.8	-0.2	0.0
1000	5.0	30	141.4	25.0	-0.4	0.0
1000	7.5	30	141.1	24.7	-0.6	0.0
1000	10.0	30	193.7	10.6	-0.4	0.0
1000	0.0	60	143.1	24.9	0.0	0.0
1000	2.5	60	142.8	24.9	-0.4	0.0
1000	5.0	60	193.9	10.8	-0.4	0.0
1000	7.5	60	141.7	24.9	-1.1	0.0
1000	10.0	60	141.8	24.6	-1.5	0.0
1000	0.0	90	143.1	24.9	0.0	0.0
1000	2.5	90	143.2	24.9	-0.4	0.0
1000	5.0	90	143.2	24.9	-0.9	0.0
1000	7.5	90	143.3	24.9	-1.3	0.0
1000	10.0	90	143.5	24.8	-1.8	0.0
1000	0.0	120	143.1	24.9	0.0	0.0
1000	2.5	120	194.0	10.9	-0.2	0.0
1000	5.0	120	144.5	24.8	-0.8	0.0
1000	7.5	120	145.1	24.7	-1.2	0.0
1000	10.0	120	146.1	24.4	-1.5	0.0
1000	0.0	150	143.1	24.9	0.0	0.0
1000	2.5	150	193.7	11.0	-0.1	0.0
1000	5.0	150	144.9	24.9	-0.5	0.0
1000	7.5	150	194.5	11.1	-0.3	0.0
1000	10.0	150	146.7	24.9	-0.9	0.0
1000	0.0	180	143.1	24.9	0.0	0.0
1000	2.5	180	144.1	25.0	-0.0	0.0
1000	5.0	180	145.1	25.0	-0.0	0.0
1000	7.5	180	146.2	25.0	-0.0	0.0
1000	10.0	180	147.2	25.0	0.0	0.0
1100	0.0	0	153.2	24.6	-0.0	0.0
1100	2.5	0	151.5	25.0	-0.0	0.0
1100	5.0	0	150.5	25.0	-0.0	0.0
1100	7.5	0	149.5	24.9	-0.0	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1100	10.0	0	194.6	12.0	-0.0	0.0
1100	0.0	30	153.2	24.6	-0.0	0.0
1100	2.5	30	151.6	25.0	-0.2	0.0
1100	5.0	30	195.1	12.2	-0.2	0.0
1100	7.5	30	150.1	24.9	-0.7	0.0
1100	10.0	30	149.3	24.8	-0.9	0.0
1100	0.0	60	153.2	24.6	-0.0	0.0
1100	2.5	60	152.1	25.0	-0.4	0.0
1100	5.0	60	151.6	24.9	-0.8	0.0
1100	7.5	60	151.2	24.8	-1.2	0.0
1100	10.0	60	150.6	24.9	-1.6	0.0
1100	0.0	90	153.2	24.6	-0.0	0.0
1100	2.5	90	153.5	24.4	-0.4	0.0
1100	5.0	90	195.3	12.4	-0.5	0.0
1100	7.5	90	153.9	24.2	-1.3	0.0
1100	10.0	90	195.2	12.4	-1.0	0.0
1100	0.0	120	153.2	24.6	-0.0	0.0
1100	2.5	120	153.9	24.5	-0.4	0.0
1100	5.0	120	154.4	24.5	-0.8	0.0
1100	7.5	120	155.1	24.5	-1.2	0.0
1100	10.0	120	155.8	24.4	-1.6	0.0
1100	0.0	150	153.2	24.6	-0.0	0.0
1100	2.5	150	153.9	24.8	-0.2	0.0
1100	5.0	150	154.9	24.7	-0.5	0.0
1100	7.5	150	156.0	24.7	-0.7	0.0
1100	10.0	150	195.7	12.8	-0.5	0.0
1100	0.0	180	153.2	24.6	-0.0	0.0
1100	2.5	180	153.9	24.8	0.0	0.0
1100	5.0	180	155.3	24.6	0.0	0.0
1100	7.5	180	156.3	24.8	0.0	0.0
1100	10.0	180	157.2	25.0	0.0	0.0
1200	0.0	0	162.0	25.0	0.0	0.0
1200	2.5	0	161.1	24.8	0.0	0.0
1200	5.0	0	160.2	24.7	-0.0	0.0
1200	7.5	0	158.5	25.0	-0.0	0.0
1200	10.0	0	195.8	13.6	-0.0	0.0
1200	0.0	30	162.0	25.0	0.0	0.0
1200	2.5	30	161.4	24.7	-0.2	0.0
1200	5.0	30	160.5	24.6	-0.5	0.0
1200	7.5	30	159.7	24.6	-0.7	0.0
1200	10.0	30	159.1	24.4	-0.9	0.0
1200	0.0	60	162.0	25.0	0.0	0.0
1200	2.5	60	161.7	24.8	-0.4	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1200	5.0	60	195.6	14.1	-0.5	0.0
1200	7.5	60	161.0	24.5	-1.2	0.0
1200	10.0	60	160.9	24.4	-1.5	0.0
1200	0.0	90	162.0	25.0	0.0	0.0
1200	2.5	90	162.2	24.9	-0.5	0.0
1200	5.0	90	162.4	24.8	-0.9	0.0
1200	7.5	90	162.1	25.0	-1.4	0.0
1200	10.0	90	162.7	24.6	-1.8	0.0
1200	0.0	120	162.0	25.0	0.0	0.0
1200	2.5	120	162.8	24.9	-0.4	0.0
1200	5.0	120	163.4	24.9	-0.8	0.0
1200	7.5	120	164.2	24.8	-1.2	0.0
1200	10.0	120	196.7	14.5	-1.0	0.0
1200	0.0	150	162.0	25.0	0.0	0.0
1200	2.5	150	163.2	24.9	-0.2	0.0
1200	5.0	150	164.2	24.9	-0.5	0.0
1200	7.5	150	196.8	14.6	-0.4	0.0
1200	10.0	150	166.7	24.9	-0.9	0.0
1200	0.0	180	162.0	25.0	0.0	0.0
1200	2.5	180	163.4	24.9	-0.0	0.0
1200	5.0	180	164.5	25.0	0.0	0.0
1200	7.5	180	165.8	25.0	0.0	0.0
1200	10.0	180	167.1	25.0	-0.0	0.0
1300	0.0	0	197.2	16.2	-0.0	0.0
1300	2.5	0	233.6	10.4	-0.0	0.0
1300	5.0	0	168.9	24.9	0.0	0.0
1300	7.5	0	167.7	24.9	0.0	0.0
1300	10.0	0	166.3	25.0	-0.0	0.0
1300	0.0	30	197.2	16.2	-0.0	0.0
1300	2.5	30	170.3	25.0	-0.2	0.0
1300	5.0	30	169.3	24.9	-0.5	0.0
1300	7.5	30	197.3	15.6	-0.5	0.0
1300	10.0	30	167.4	24.8	-0.9	0.0
1300	0.0	60	197.2	16.2	-0.0	0.0
1300	2.5	60	170.7	25.0	-0.4	0.0
1300	5.0	60	170.2	24.9	-0.8	0.0
1300	7.5	60	197.4	15.8	-0.8	0.0
1300	10.0	60	169.2	24.8	-1.6	0.0
1300	0.0	90	197.2	16.2	-0.0	0.0
1300	2.5	90	197.5	16.1	-0.3	0.0
1300	5.0	90	197.6	16.1	-0.6	0.0
1300	7.5	90	171.7	24.8	-1.4	0.0
1300	10.0	90	172.1	24.7	-1.9	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1300	0.0	120	197.2	16.2	-0.0	0.0
1300	2.5	120	172.1	25.0	-0.4	0.0
1300	5.0	120	172.9	24.9	-0.8	0.0
1300	7.5	120	197.8	16.4	-0.9	0.0
1300	10.0	120	176.9	23.6	-1.6	0.0
1300	0.0	150	197.2	16.2	-0.0	0.0
1300	2.5	150	172.6	25.0	-0.2	0.0
1300	5.0	150	177.7	23.0	-0.4	0.0
1300	7.5	150	177.4	23.7	-0.7	0.0
1300	10.0	150	177.8	24.1	-0.9	0.0
1300	0.0	180	197.2	16.2	-0.0	0.0
1300	2.5	180	172.8	25.0	-0.0	0.0
1300	5.0	180	176.9	23.5	-0.0	0.0
1300	7.5	180	177.6	23.9	0.0	0.0
1300	10.0	180	177.5	24.8	-0.0	0.0
1400	0.0	0	199.0	18.2	-0.0	0.0
1400	2.5	0	182.4	23.5	0.0	0.0
1400	5.0	0	178.0	25.0	-0.0	0.0
1400	7.5	0	176.6	25.0	0.0	0.0
1400	10.0	0	175.3	25.0	0.0	0.0
1400	0.0	30	199.0	18.2	-0.0	0.0
1400	2.5	30	182.8	23.4	-0.2	0.0
1400	5.0	30	183.4	22.7	-0.4	0.0
1400	7.5	30	177.4	24.9	-0.7	0.0
1400	10.0	30	198.5	17.3	-0.7	0.0
1400	0.0	60	199.0	18.2	-0.0	0.0
1400	2.5	60	199.0	18.1	-0.3	0.0
1400	5.0	60	198.9	17.9	-0.6	0.0
1400	7.5	60	198.8	17.8	-0.9	0.0
1400	10.0	60	198.7	17.7	-1.2	0.0
1400	0.0	90	199.0	18.2	-0.0	0.0
1400	2.5	90	198.9	18.2	-0.4	0.0
1400	5.0	90	198.9	18.3	-0.7	0.0
1400	7.5	90	183.4	23.7	-1.4	0.0
1400	10.0	90	183.9	23.6	-1.8	0.0
1400	0.0	120	199.0	18.2	-0.0	0.0
1400	2.5	120	182.1	24.7	-0.4	0.0
1400	5.0	120	198.6	18.7	-0.6	0.0
1400	7.5	120	183.8	24.7	-1.2	0.0
1400	10.0	120	198.7	19.1	-1.3	0.0
1400	0.0	150	199.0	18.2	-0.0	0.0
1400	2.5	150	198.8	18.6	-0.2	0.0
1400	5.0	150	183.6	24.9	-0.5	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1400	7.5	150	199.4	19.1	-0.6	0.0
1400	10.0	150	188.2	23.9	-0.9	0.0
1400	0.0	180	199.0	18.2	-0.0	0.0
1400	2.5	180	183.2	24.5	0.0	0.0
1400	5.0	180	198.2	19.2	0.0	0.0
1400	7.5	180	187.6	23.8	0.0	0.0
1400	10.0	180	188.7	24.1	-0.0	0.0
1500	0.0	0	201.0	20.6	0.0	0.0
1500	2.5	0	188.8	25.0	0.0	0.0
1500	5.0	0	200.0	20.0	-0.0	0.0
1500	7.5	0	199.6	19.8	0.0	0.0
1500	10.0	0	188.1	23.3	0.0	0.0
1500	0.0	30	201.0	20.6	0.0	0.0
1500	2.5	30	200.5	20.4	-0.2	0.0
1500	5.0	30	188.2	24.7	-0.5	0.0
1500	7.5	30	186.8	24.8	-0.7	0.0
1500	10.0	30	199.7	19.6	-0.8	0.0
1500	0.0	60	201.0	20.6	0.0	0.0
1500	2.5	60	189.6	25.0	-0.4	0.0
1500	5.0	60	189.0	24.9	-0.8	0.0
1500	7.5	60	200.0	20.3	-1.0	0.0
1500	10.0	60	200.0	20.1	-1.4	0.0
1500	0.0	90	201.0	20.6	0.0	0.0
1500	2.5	90	201.3	20.5	-0.4	0.0
1500	5.0	90	193.4	23.5	-0.9	0.0
1500	7.5	90	193.7	23.4	-1.4	0.0
1500	10.0	90	193.8	23.4	-1.8	0.0
1500	0.0	120	201.0	20.6	0.0	0.0
1500	2.5	120	201.4	20.7	-0.4	0.0
1500	5.0	120	201.8	20.8	-0.7	0.0
1500	7.5	120	194.3	24.2	-1.2	0.0
1500	10.0	120	195.8	24.0	-1.6	0.0
1500	0.0	150	201.0	20.6	0.0	0.0
1500	2.5	150	194.1	23.8	-0.2	0.0
1500	5.0	150	201.1	21.5	-0.4	0.0
1500	7.5	150	196.2	24.2	-0.7	0.0
1500	10.0	150	197.6	24.2	-1.0	0.0
1500	0.0	180	201.0	20.6	0.0	0.0
1500	2.5	180	201.7	20.9	-0.0	0.0
1500	5.0	180	195.2	24.2	-0.0	0.0
1500	7.5	180	196.2	24.5	-0.0	0.0
1500	10.0	180	203.2	22.0	-0.0	0.0

Air Density = 1.2 kg/m^3

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1000	0.0	0	145.6	25.0	0.0	0.0
1000	2.5	0	144.6	24.9	0.0	0.0
1000	5.0	0	143.4	25.0	0.0	0.0
1000	7.5	0	142.5	24.9	-0.0	0.0
1000	10.0	0	141.3	25.0	0.0	0.0
1000	0.0	30	145.6	25.0	0.0	0.0
1000	2.5	30	144.7	24.9	-0.2	0.0
1000	5.0	30	143.7	25.0	-0.5	0.0
1000	7.5	30	194.0	10.9	-0.4	0.0
1000	10.0	30	191.9	11.1	-0.5	0.0
1000	0.0	60	145.6	25.0	0.0	0.0
1000	2.5	60	145.1	25.0	-0.4	0.0
1000	5.0	60	144.7	24.9	-0.8	0.0
1000	7.5	60	144.6	24.7	-1.2	0.0
1000	10.0	60	143.9	24.8	-1.6	0.0
1000	0.0	90	145.6	25.0	0.0	0.0
1000	2.5	90	145.7	24.9	-0.5	0.0
1000	5.0	90	194.5	11.1	-0.5	0.0
1000	7.5	90	145.7	24.9	-1.4	0.0
1000	10.0	90	194.5	11.1	-1.0	0.0
1000	0.0	120	145.6	25.0	0.0	0.0
1000	2.5	120	146.2	25.0	-0.4	0.0
1000	5.0	120	146.8	24.9	-0.8	0.0
1000	7.5	120	147.5	24.9	-1.3	0.0
1000	10.0	120	149.1	24.3	-1.7	0.0
1000	0.0	150	145.6	25.0	0.0	0.0
1000	2.5	150	146.6	25.0	-0.2	0.0
1000	5.0	150	148.2	24.6	-0.5	0.0
1000	7.5	150	193.2	11.6	-0.4	0.0
1000	10.0	150	150.0	24.7	-1.0	0.0
1000	0.0	180	145.6	25.0	0.0	0.0
1000	2.5	180	146.7	25.0	-0.0	0.0
1000	5.0	180	148.3	24.7	-0.0	0.0
1000	7.5	180	149.0	25.0	0.0	0.0
1000	10.0	180	193.7	11.7	0.0	0.0
1100	0.0	0	155.8	24.8	0.0	0.0
1100	2.5	0	154.8	24.7	0.0	0.0
1100	5.0	0	153.2	25.0	0.0	0.0
1100	7.5	0	152.0	25.0	-0.0	0.0
1100	10.0	0	150.9	25.0	0.0	0.0
1100	0.0	30	155.8	24.8	0.0	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1100	2.5	30	155.0	24.7	-0.2	0.0
1100	5.0	30	195.1	12.6	-0.3	0.0
1100	7.5	30	153.4	24.5	-0.7	0.0
1100	10.0	30	151.6	24.9	-0.9	0.0
1100	0.0	60	155.8	24.8	0.0	0.0
1100	2.5	60	195.4	12.8	-0.2	0.0
1100	5.0	60	155.0	24.6	-0.8	0.0
1100	7.5	60	154.9	24.4	-1.2	0.0
1100	10.0	60	195.4	12.6	-0.9	0.0
1100	0.0	90	155.8	24.8	0.0	0.0
1100	2.5	90	195.7	12.8	-0.3	0.0
1100	5.0	90	156.0	24.7	-1.0	0.0
1100	7.5	90	156.2	24.6	-1.5	0.0
1100	10.0	90	156.3	24.6	-1.9	0.0
1100	0.0	120	155.8	24.8	0.0	0.0
1100	2.5	120	156.1	25.0	-0.4	0.0
1100	5.0	120	195.9	12.9	-0.5	0.0
1100	7.5	120	157.7	24.8	-1.3	0.0
1100	10.0	120	158.5	24.8	-1.7	0.0
1100	0.0	150	155.8	24.8	0.0	0.0
1100	2.5	150	156.9	24.8	-0.2	0.0
1100	5.0	150	157.9	24.9	-0.5	0.0
1100	7.5	150	196.1	13.1	-0.4	0.0
1100	10.0	150	196.2	13.3	-0.6	0.0
1100	0.0	180	155.8	24.8	0.0	0.0
1100	2.5	180	156.8	25.0	-0.0	0.0
1100	5.0	180	158.2	24.9	-0.0	0.0
1100	7.5	180	159.4	25.0	-0.0	0.0
1100	10.0	180	160.8	25.0	0.0	0.0
1200	0.0	0	196.9	14.7	-0.0	0.0
1200	2.5	0	196.7	14.5	0.0	0.0
1200	5.0	0	163.0	24.9	-0.0	0.0
1200	7.5	0	196.5	14.2	0.0	0.0
1200	10.0	0	160.7	24.8	-0.0	0.0
1200	0.0	30	196.9	14.7	-0.0	0.0
1200	2.5	30	164.3	25.0	-0.2	0.0
1200	5.0	30	163.3	24.9	-0.5	0.0
1200	7.5	30	196.5	14.2	-0.5	0.0
1200	10.0	30	196.4	14.1	-0.6	0.0
1200	0.0	60	196.9	14.7	-0.0	0.0
1200	2.5	60	165.0	24.9	-0.4	0.0
1200	5.0	60	164.3	24.9	-0.9	0.0
1200	7.5	60	163.8	24.8	-1.3	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1200	10.0	60	196.6	14.3	-1.1	0.0
1200	0.0	90	196.9	14.7	-0.0	0.0
1200	2.5	90	165.6	24.9	-0.5	0.0
1200	5.0	90	196.9	14.7	-0.6	0.0
1200	7.5	90	196.9	14.7	-0.9	0.0
1200	10.0	90	165.8	24.9	-2.0	0.0
1200	0.0	120	196.9	14.7	-0.0	0.0
1200	2.5	120	197.0	14.7	-0.3	0.0
1200	5.0	120	167.0	24.9	-0.9	0.0
1200	7.5	120	167.8	24.9	-1.3	0.0
1200	10.0	120	170.5	23.8	-1.7	0.0
1200	0.0	150	196.9	14.7	-0.0	0.0
1200	2.5	150	197.0	14.8	-0.2	0.0
1200	5.0	150	168.0	24.9	-0.5	0.0
1200	7.5	150	170.9	24.0	-0.7	0.0
1200	10.0	150	171.7	24.3	-1.0	0.0
1200	0.0	180	196.9	14.7	-0.0	0.0
1200	2.5	180	166.9	25.0	0.0	0.0
1200	5.0	180	168.4	25.0	0.0	0.0
1200	7.5	180	196.8	15.4	-0.0	0.0
1200	10.0	180	196.0	15.8	-0.0	0.0
1300	0.0	0	177.3	24.0	-0.0	0.0
1300	2.5	0	176.5	23.6	0.0	0.0
1300	5.0	0	172.5	25.0	0.0	0.0
1300	7.5	0	171.1	25.0	0.0	0.0
1300	10.0	0	169.8	25.0	0.0	0.0
1300	0.0	30	177.3	24.0	-0.0	0.0
1300	2.5	30	197.9	16.6	-0.2	0.0
1300	5.0	30	197.7	16.4	-0.3	0.0
1300	7.5	30	171.7	24.9	-0.7	0.0
1300	10.0	30	170.6	24.9	-1.0	0.0
1300	0.0	60	177.3	24.0	-0.0	0.0
1300	2.5	60	197.7	16.8	-0.3	0.0
1300	5.0	60	176.2	23.8	-0.8	0.0
1300	7.5	60	173.6	24.8	-1.3	0.0
1300	10.0	60	197.9	16.3	-1.2	0.0
1300	0.0	90	177.3	24.0	-0.0	0.0
1300	2.5	90	177.2	24.0	-0.5	0.0
1300	5.0	90	177.4	24.0	-1.0	0.0
1300	7.5	90	177.4	24.0	-1.5	0.0
1300	10.0	90	177.9	23.8	-1.9	0.0
1300	0.0	120	177.3	24.0	-0.0	0.0
1300	2.5	120	177.7	24.1	-0.4	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1300	5.0	120	197.3	17.3	-0.6	0.0
1300	7.5	120	198.5	17.2	-1.0	0.0
1300	10.0	120	179.2	24.7	-1.8	0.0
1300	0.0	150	177.3	24.0	-0.0	0.0
1300	2.5	150	178.4	24.1	-0.2	0.0
1300	5.0	150	178.3	24.9	-0.5	0.0
1300	7.5	150	179.8	24.9	-0.8	0.0
1300	10.0	150	182.3	24.2	-1.0	0.0
1300	0.0	180	177.3	24.0	-0.0	0.0
1300	2.5	180	177.4	24.8	0.0	0.0
1300	5.0	180	178.6	25.0	-0.0	0.0
1300	7.5	180	181.6	24.2	-0.0	0.0
1300	10.0	180	199.0	17.9	-0.0	0.0
1400	0.0	0	199.6	19.3	-0.0	0.0
1400	2.5	0	183.8	25.0	-0.0	0.0
1400	5.0	0	182.7	24.7	-0.0	0.0
1400	7.5	0	183.0	23.8	0.0	0.0
1400	10.0	0	182.4	23.4	0.0	0.0
1400	0.0	30	199.6	19.3	-0.0	0.0
1400	2.5	30	184.2	24.9	-0.3	0.0
1400	5.0	30	198.8	18.8	-0.4	0.0
1400	7.5	30	183.4	23.9	-0.7	0.0
1400	10.0	30	199.0	18.1	-0.8	0.0
1400	0.0	60	199.6	19.3	-0.0	0.0
1400	2.5	60	199.4	19.1	-0.4	0.0
1400	5.0	60	184.0	24.9	-0.9	0.0
1400	7.5	60	183.4	24.9	-1.3	0.0
1400	10.0	60	182.9	24.7	-1.7	0.0
1400	0.0	90	199.6	19.3	-0.0	0.0
1400	2.5	90	187.5	24.0	-0.5	0.0
1400	5.0	90	199.6	19.3	-0.8	0.0
1400	7.5	90	199.6	19.3	-1.2	0.0
1400	10.0	90	188.0	23.8	-2.0	0.0
1400	0.0	120	199.6	19.3	-0.0	0.0
1400	2.5	120	188.0	24.1	-0.4	0.0
1400	5.0	120	199.8	19.7	-0.7	0.0
1400	7.5	120	189.6	24.2	-1.3	0.0
1400	10.0	120	190.5	24.2	-1.8	0.0
1400	0.0	150	199.6	19.3	-0.0	0.0
1400	2.5	150	188.6	24.1	-0.3	0.0
1400	5.0	150	189.7	24.3	-0.5	0.0
1400	7.5	150	200.5	20.3	-0.7	0.0
1400	10.0	150	192.5	24.5	-1.0	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1400	0.0	180	199.6	19.3	-0.0	0.0
1400	2.5	180	188.7	24.1	0.0	0.0
1400	5.0	180	190.0	24.4	0.0	0.0
1400	7.5	180	191.4	24.6	0.0	0.0
1400	10.0	180	193.0	24.8	-0.0	0.0
1500	0.0	0	245.7	12.4	-0.0	0.0
1500	2.5	0	201.3	21.7	-0.0	0.0
1500	5.0	0	194.3	23.9	0.0	0.0
1500	7.5	0	201.2	20.6	-0.0	0.0
1500	10.0	0	200.4	20.4	0.0	0.0
1500	0.0	30	245.7	12.4	-0.0	0.0
1500	2.5	30	195.8	24.1	-0.3	0.0
1500	5.0	30	200.8	21.4	-0.4	0.0
1500	7.5	30	201.5	20.7	-0.7	0.0
1500	10.0	30	189.7	25.0	-1.0	0.0
1500	0.0	60	245.7	12.4	-0.0	0.0
1500	2.5	60	202.0	21.7	-0.4	0.0
1500	5.0	60	195.6	24.1	-0.9	0.0
1500	7.5	60	195.3	23.9	-1.3	0.0
1500	10.0	60	194.9	23.7	-1.7	0.0
1500	0.0	90	245.7	12.4	-0.0	0.0
1500	2.5	90	196.9	24.3	-0.5	0.0
1500	5.0	90	203.2	21.6	-0.9	0.0
1500	7.5	90	202.6	21.8	-1.4	0.0
1500	10.0	90	197.1	24.3	-2.0	0.0
1500	0.0	120	245.7	12.4	-0.0	0.0
1500	2.5	120	197.4	24.5	-0.4	0.0
1500	5.0	120	198.4	24.5	-0.9	0.0
1500	7.5	120	204.7	22.0	-1.2	0.0
1500	10.0	120	204.1	22.7	-1.7	0.0
1500	0.0	150	245.7	12.4	-0.0	0.0
1500	2.5	150	203.3	22.1	-0.2	0.0
1500	5.0	150	204.3	22.3	-0.5	0.0
1500	7.5	150	201.2	24.6	-0.8	0.0
1500	10.0	150	202.8	24.7	-1.1	0.0
1500	0.0	180	245.7	12.4	-0.0	0.0
1500	2.5	180	198.2	24.6	0.0	0.0
1500	5.0	180	204.3	22.5	-0.0	0.0
1500	7.5	180	201.8	24.7	0.0	0.0
1500	10.0	180	203.5	24.9	0.0	0.0

Air Density = 1.3 kg/m^3

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1000	0.0	0	148.9	25.0	0.0	0.0
1000	2.5	0	194.6	11.5	0.0	0.0
1000	5.0	0	146.4	25.0	-0.0	0.0
1000	7.5	0	145.2	25.0	0.0	0.0
1000	10.0	0	144.2	24.9	0.0	0.0
1000	0.0	30	148.9	25.0	0.0	0.0
1000	2.5	30	193.4	11.7	-0.1	0.0
1000	5.0	30	146.8	25.0	-0.5	0.0
1000	7.5	30	192.7	11.5	-0.4	0.0
1000	10.0	30	144.8	24.9	-1.0	0.0
1000	0.0	60	148.9	25.0	0.0	0.0
1000	2.5	60	193.2	11.7	-0.2	0.0
1000	5.0	60	194.6	11.5	-0.5	0.0
1000	7.5	60	147.3	24.8	-1.3	0.0
1000	10.0	60	192.0	11.7	-0.9	0.0
1000	0.0	90	148.9	25.0	0.0	0.0
1000	2.5	90	149.1	24.8	-0.5	0.0
1000	5.0	90	149.3	24.7	-1.0	0.0
1000	7.5	90	149.6	24.5	-1.5	0.0
1000	10.0	90	149.9	24.4	-2.0	0.0
1000	0.0	120	148.9	25.0	0.0	0.0
1000	2.5	120	149.7	24.8	-0.4	0.0
1000	5.0	120	150.3	24.9	-0.9	0.0
1000	7.5	120	151.1	24.8	-1.4	0.0
1000	10.0	120	151.6	25.0	-1.8	0.0
1000	0.0	150	148.9	25.0	0.0	0.0
1000	2.5	150	150.1	24.9	-0.3	0.0
1000	5.0	150	151.4	24.8	-0.5	0.0
1000	7.5	150	195.4	11.8	-0.4	0.0
1000	10.0	150	153.6	24.9	-1.1	0.0
1000	0.0	180	148.9	25.0	0.0	0.0
1000	2.5	180	150.3	24.9	0.0	0.0
1000	5.0	180	151.5	25.0	-0.0	0.0
1000	7.5	180	195.4	11.9	-0.0	0.0
1000	10.0	180	154.2	25.0	0.0	0.0
1100	0.0	0	159.3	25.0	-0.0	0.0
1100	2.5	0	196.1	13.2	-0.0	0.0
1100	5.0	0	156.7	24.9	0.0	0.0
1100	7.5	0	155.5	24.9	-0.0	0.0
1100	10.0	0	154.4	24.8	0.0	0.0
1100	0.0	30	159.3	25.0	-0.0	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1100	2.5	30	196.2	13.2	-0.2	0.0
1100	5.0	30	196.0	13.1	-0.3	0.0
1100	7.5	30	156.2	24.8	-0.8	0.0
1100	10.0	30	195.8	12.8	-0.6	0.0
1100	0.0	60	159.3	25.0	-0.0	0.0
1100	2.5	60	196.2	13.3	-0.3	0.0
1100	5.0	60	196.1	13.2	-0.5	0.0
1100	7.5	60	196.1	13.1	-0.8	0.0
1100	10.0	60	196.0	13.1	-1.0	0.0
1100	0.0	90	159.3	25.0	-0.0	0.0
1100	2.5	90	196.3	13.3	-0.3	0.0
1100	5.0	90	196.3	13.3	-0.6	0.0
1100	7.5	90	196.2	13.4	-0.9	0.0
1100	10.0	90	196.2	13.4	-1.2	0.0
1100	0.0	120	159.3	25.0	-0.0	0.0
1100	2.5	120	196.3	13.4	-0.3	0.0
1100	5.0	120	160.8	24.9	-0.9	0.0
1100	7.5	120	161.6	24.9	-1.4	0.0
1100	10.0	120	162.6	24.8	-1.9	0.0
1100	0.0	150	159.3	25.0	-0.0	0.0
1100	2.5	150	160.6	25.0	-0.3	0.0
1100	5.0	150	161.9	25.0	-0.5	0.0
1100	7.5	150	164.5	24.1	-0.8	0.0
1100	10.0	150	196.2	14.0	-0.7	0.0
1100	0.0	180	159.3	25.0	-0.0	0.0
1100	2.5	180	160.7	25.0	-0.0	0.0
1100	5.0	180	162.2	25.0	-0.0	0.0
1100	7.5	180	164.8	24.3	-0.0	0.0
1100	10.0	180	165.4	24.9	-0.0	0.0
1200	0.0	0	171.4	24.1	-0.0	0.0
1200	2.5	0	196.9	15.3	-0.0	0.0
1200	5.0	0	166.8	25.0	0.0	0.0
1200	7.5	0	197.1	14.8	0.0	0.0
1200	10.0	0	163.9	25.0	0.0	0.0
1200	0.0	30	171.4	24.1	-0.0	0.0
1200	2.5	30	170.6	23.8	-0.3	0.0
1200	5.0	30	167.2	25.0	-0.5	0.0
1200	7.5	30	166.1	24.9	-0.8	0.0
1200	10.0	30	197.1	14.7	-0.7	0.0
1200	0.0	60	171.4	24.1	-0.0	0.0
1200	2.5	60	170.7	24.1	-0.4	0.0
1200	5.0	60	196.6	15.4	-0.6	0.0
1200	7.5	60	167.8	24.9	-1.4	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1200	10.0	60	196.8	15.1	-1.2	0.0
1200	0.0	90	171.4	24.1	-0.0	0.0
1200	2.5	90	171.2	24.2	-0.5	0.0
1200	5.0	90	171.0	24.3	-1.0	0.0
1200	7.5	90	171.4	24.2	-1.6	0.0
1200	10.0	90	171.7	24.0	-2.1	0.0
1200	0.0	120	171.4	24.1	-0.0	0.0
1200	2.5	120	197.5	15.5	-0.3	0.0
1200	5.0	120	171.7	24.9	-0.9	0.0
1200	7.5	120	172.3	25.0	-1.4	0.0
1200	10.0	120	173.4	24.9	-1.9	0.0
1200	0.0	150	171.4	24.1	-0.0	0.0
1200	2.5	150	197.7	15.6	-0.2	0.0
1200	5.0	150	197.8	15.8	-0.4	0.0
1200	7.5	150	197.8	16.1	-0.6	0.0
1200	10.0	150	197.7	16.4	-0.8	0.0
1200	0.0	180	171.4	24.1	-0.0	0.0
1200	2.5	180	171.5	25.0	-0.0	0.0
1200	5.0	180	173.1	25.0	-0.0	0.0
1200	7.5	180	175.7	24.4	0.0	0.0
1200	10.0	180	177.2	24.6	0.0	0.0
1300	0.0	0	181.9	24.2	-0.0	0.0
1300	2.5	0	198.7	17.5	0.0	0.0
1300	5.0	0	177.4	24.8	0.0	0.0
1300	7.5	0	197.0	17.3	0.0	0.0
1300	10.0	0	197.8	16.8	-0.0	0.0
1300	0.0	30	181.9	24.2	-0.0	0.0
1300	2.5	30	179.0	24.9	-0.3	0.0
1300	5.0	30	177.6	25.0	-0.5	0.0
1300	7.5	30	177.8	24.1	-0.8	0.0
1300	10.0	30	197.6	17.0	-0.8	0.0
1300	0.0	60	181.9	24.2	-0.0	0.0
1300	2.5	60	198.6	17.7	-0.4	0.0
1300	5.0	60	178.9	24.9	-0.9	0.0
1300	7.5	60	178.3	24.8	-1.4	0.0
1300	10.0	60	177.7	24.7	-1.8	0.0
1300	0.0	90	181.9	24.2	-0.0	0.0
1300	2.5	90	182.7	23.8	-0.5	0.0
1300	5.0	90	182.1	24.1	-1.1	0.0
1300	7.5	90	182.2	24.1	-1.6	0.0
1300	10.0	90	182.5	23.9	-2.1	0.0
1300	0.0	120	181.9	24.2	-0.0	0.0
1300	2.5	120	182.5	24.3	-0.5	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1300	5.0	120	199.1	18.2	-0.7	0.0
1300	7.5	120	199.0	18.5	-1.1	0.0
1300	10.0	120	185.2	24.4	-1.9	0.0
1300	0.0	150	181.9	24.2	-0.0	0.0
1300	2.5	150	183.1	24.4	-0.3	0.0
1300	5.0	150	184.3	24.5	-0.5	0.0
1300	7.5	150	185.7	24.7	-0.8	0.0
1300	10.0	150	187.4	24.7	-1.1	0.0
1300	0.0	180	181.9	24.2	-0.0	0.0
1300	2.5	180	183.3	24.4	-0.0	0.0
1300	5.0	180	184.7	24.6	-0.0	0.0
1300	7.5	180	199.5	19.0	0.0	0.0
1300	10.0	180	187.9	25.0	-0.0	0.0
1400	0.0	0	191.8	24.6	0.0	0.0
1400	2.5	0	190.4	24.4	-0.0	0.0
1400	5.0	0	189.0	24.2	-0.0	0.0
1400	7.5	0	187.8	23.9	0.0	0.0
1400	10.0	0	183.9	25.0	-0.0	0.0
1400	0.0	30	191.8	24.6	0.0	0.0
1400	2.5	30	191.0	24.2	-0.3	0.0
1400	5.0	30	189.5	24.2	-0.5	0.0
1400	7.5	30	188.4	24.0	-0.8	0.0
1400	10.0	30	185.0	24.9	-1.1	0.0
1400	0.0	60	191.8	24.6	0.0	0.0
1400	2.5	60	200.8	20.4	-0.4	0.0
1400	5.0	60	200.4	20.3	-0.8	0.0
1400	7.5	60	190.1	24.1	-1.4	0.0
1400	10.0	60	200.1	19.9	-1.6	0.0
1400	0.0	90	191.8	24.6	0.0	0.0
1400	2.5	90	201.1	20.6	-0.5	0.0
1400	5.0	90	192.2	24.4	-1.1	0.0
1400	7.5	90	192.3	24.4	-1.6	0.0
1400	10.0	90	192.8	24.2	-2.2	0.0
1400	0.0	120	191.8	24.6	0.0	0.0
1400	2.5	120	192.5	24.8	-0.5	0.0
1400	5.0	120	201.7	21.0	-0.8	0.0
1400	7.5	120	194.7	24.7	-1.4	0.0
1400	10.0	120	195.8	24.7	-1.9	0.0
1400	0.0	150	191.8	24.6	0.0	0.0
1400	2.5	150	193.3	24.7	-0.3	0.0
1400	5.0	150	194.6	25.0	-0.6	0.0
1400	7.5	150	203.0	21.6	-0.7	0.0
1400	10.0	150	202.9	22.4	-1.0	0.0

Range (m)	Wind (m/s)	Wind Dir (°)	Velocity (m/s)	Elevation (°)	Azimuth (°)	Error (m)
1400	0.0	180	191.8	24.6	0.0	0.0
1400	2.5	180	193.2	24.9	-0.0	0.0
1400	5.0	180	195.1	25.0	0.0	0.0
1400	7.5	180	197.4	24.9	0.0	0.0
1400	10.0	180	199.4	25.0	-0.0	0.0
1500	0.0	0	203.0	24.4	0.0	0.0
1500	2.5	0	201.7	24.1	0.0	0.0
1500	5.0	0	203.8	22.3	0.0	0.0
1500	7.5	0	197.4	24.3	0.0	0.0
1500	10.0	0	195.9	24.1	0.0	0.0
1500	0.0	30	203.0	24.4	0.0	0.0
1500	2.5	30	201.4	24.3	-0.3	0.0
1500	5.0	30	199.0	24.7	-0.5	0.0
1500	7.5	30	198.1	24.3	-0.8	0.0
1500	10.0	30	202.8	21.7	-1.0	0.0
1500	0.0	60	203.0	24.4	0.0	0.0
1500	2.5	60	202.7	24.1	-0.5	0.0
1500	5.0	60	205.3	22.5	-0.9	0.0
1500	7.5	60	199.9	24.5	-1.4	0.0
1500	10.0	60	203.8	22.4	-1.7	0.0
1500	0.0	90	203.0	24.4	0.0	0.0
1500	2.5	90	202.4	24.8	-0.6	0.0
1500	5.0	90	202.6	24.7	-1.1	0.0
1500	7.5	90	202.9	24.6	-1.7	0.0
1500	10.0	90	205.9	23.1	-2.1	0.0
1500	0.0	120	203.0	24.4	0.0	0.0
1500	2.5	120	203.4	24.8	-0.5	0.0
1500	5.0	120	204.4	24.8	-1.0	0.0
1500	7.5	120	205.6	24.8	-1.5	0.0
1500	10.0	120	207.4	24.5	-1.9	0.0
1500	0.0	150	203.0	24.4	0.0	0.0
1500	2.5	150	204.1	24.8	-0.3	0.0
1500	5.0	150	206.5	24.5	-0.6	0.0
1500	7.5	150	208.4	24.5	-0.8	0.0
1500	10.0	150	209.8	24.9	-1.2	0.0
1500	0.0	180	203.0	24.4	0.0	0.0
1500	2.5	180	204.5	24.8	0.0	0.0
1500	5.0	180	206.4	24.9	-0.0	0.0
1500	7.5	180	208.6	25.0	0.0	0.0
1500	10.0	180	210.9	25.0	0.0	0.0

Code

Here are simulation programs we used in our model as follow.

Solve differential equations:

```
import numpy as np
import matplotlib.pyplot as plt
import json
import csv
import os
from scipy.integrate import solve_ivp
from scipy.optimize import minimize, differential_evolution, Bounds

class ArtillerySolver3D:
    def __init__(self):
        # Physical constants
        self.mass = 5.0                    # Projectile mass [kg]
        self.diameter = 0.11               # Projectile diameter [m]
        self.radius = self.diameter / 2    # Projectile radius [m]
        self.area = np.pi * self.radius**2 # Cross-sectional area [m  ]
        self.drag_coeff = 0.47            # Sphere drag coefficient
        # Reference sea-level density (can be swept per-run). Actual density
        # used during simulation varies with altitude via get_air_density().
        self.air_density = 1.2             # kept for backward-compat; use
            ↪ sea_level_density for profiles
        self.sea_level_density = float(self.air_density)
        # Simple exponential scale-height model (rho(z) = rho0 * exp(-z/
            ↪ scale_height)).
        # Scale height ~8500 m is a reasonable approximation for troposphere.
        self.scale_height = 8500.0  # meters
        # Optional: user may assign a callable self.air_density_func(z) to
            ↪ override the simple model.
        self.air_density_func = None
        self.gravity = 9.807                 # Gravity [m/s  ]
        # Speed of sound used to compute Mach number for high-speed drag model
            ↪  [m/s]
        # Use 340 m/s as a practical threshold (close to standard sea-level
            ↪ value)
        self.speed_of_sound = 340.0
        self.k_drag = 0.5 * self.air_density * self.drag_coeff * self.area

        # Cannon constraints
        self.max_muzzle_velocity = 450     # Maximum muzzle velocity [m/s]

        # Create a robust ground-collision event (bound function with
            ↪ attributes)
        def _ground_collision_event(t, state, *args):
            return state[2]

        _ground_collision_event.terminal = True
        _ground_collision_event.direction = -1
        self.ground_collision_event = _ground_collision_event
    def get_air_density_at_height(self, z):
        """Return air density (kg/m^3) at altitude z (meters).
```

```
        By default this uses a simple exponential scale-height model anchored
            ↪ at
        self.sea_level_density. If self.air_density_func is set to a callable,
        that will be used instead (useful for high-fidelity profiles).
        """
        # ensure numeric
        try:
            zval = float(z)
        except Exception:
            zval = 0.0

        if self.air_density_func is not None:
            try:
                return float(self.air_density_func(zval))
            except Exception:
                # fallback to exponential model on error
                pass

        # clamp: do not allow extremely negative altitudes to blow up density
        if zval < 0.0:
            zval = 0.0

        rho0 = float(self.sea_level_density)
        H = max(100.0, float(self.scale_height))
        return rho0 * np.exp(-zval / H)

    def wind_force(self, velocity, wind_velocity, altitude=0.0):
        """Calculate aerodynamic wind/drag force on projectile at given
            ↪ altitude.

        Parameters:
        - velocity: projectile velocity vector (3,)
        - wind_velocity: air/wind velocity vector (3,)
        - altitude: altitude z (m) at which to evaluate air density
        """
        relative_velocity = velocity - wind_velocity
        speed = np.linalg.norm(relative_velocity)

        if speed < 1e-10:
            return np.zeros(3)

        # Select drag-coefficient model depending on speed
        if speed <= 340.0:
            cd = self.drag_coeff_for_speed(speed)
        else:
            mach = speed / float(self.speed_of_sound)
            cd = self.drag_coeff_for_mach(mach)

        # Evaluate local air density at altitude
        rho_local = self.get_air_density_at_height(altitude)

        # Drag force magnitude (quadratic in speed)
        drag_magnitude = 0.5 * rho_local * cd * self.area * speed**2
```

```
        # Direction of drag force (opposite to relative velocity)
        drag_direction = -relative_velocity / speed

        return drag_magnitude * drag_direction

    def drag_coeff_for_mach(self, mach):
            """Estimate drag coefficient Cd as a function of Mach number.

            This is a simple piecewise-linear approximation intended to
               ↪ capture the
            transonic/supersonic rise in drag coefficient near Ma~1 and
               ↪ then a
            gradual decline at higher Mach numbers. Anchors are tunable.

            NOTES/ASSUMPTIONS:
            - We use a pragmatic anchor set because a full compressible-
               ↪ flow Cd(Ma)
                curve requires experimental data and depends on projectile
                   ↪  shape.
            - Anchors (Ma -> Cd): 0.95->0.50, 1.00->1.20 (transonic spike)
               ↪ ,
                1.20->0.90, 2.00->0.50, 5.00->0.30
            - If you have measured Cd vs Ma for your projectile, replace
               ↪ these
                anchor points with that dataset.
            """
            m = max(0.0, float(mach))
            # piecewise linear interpolation over Mach anchors
            return float(np.interp(m, [0.95, 1.00, 1.20, 2.00, 5.00],
               ↪ [0.50, 1.20, 0.90, 0.50, 0.30]))

    def drag_coeff_for_speed(self, speed):
        """Return an estimated drag coefficient Cd as a function of relative
           ↪ speed.

        This uses a simple piecewise-linear interpolation between three anchor
           ↪  points
        to approximate the effect of changing flow/turbulence regimes.
        Tunable anchor points: (0 m/s -> 0.9), (150 m/s -> 0.47), (500 m/s ->
           ↪ 0.1).
        """
        # clamp speed to non-negative
        s = max(0.0, float(speed))
        # piecewise linear interpolation
        return float(np.interp(s, [0.0, 150.0, 500.0], [0.9, 0.47, 0.1]))

    def projectile_dynamics(self, t, state, muzzle_velocity, elevation_angle,
       ↪ azimuth_angle, wind_velocity):
        """
        3D Projectile motion equations with wind effects
        state = [x, y, z, vx, vy, vz]
        """
        position = state[:3]
```

```
        velocity = state[3:]

        # Convert angles to radians
        elevation_rad = np.radians(elevation_angle)
        azimuth_rad = np.radians(azimuth_angle)

        # Initialize acceleration
        acceleration = np.zeros(3)

        # Gravity (only in z-direction)
        acceleration[2] = -self.gravity

        # Aerodynamic drag (always apply; wind_velocity is the air speed
            ↪ vector)
        # Use current altitude (state[2]) to evaluate local air density
        wind_force_vec = self.wind_force(velocity, wind_velocity, altitude=
            ↪ position[2])
        acceleration += wind_force_vec / self.mass

        # Combine derivatives
        derivatives = np.concatenate([velocity, acceleration])

        return derivatives

    def ground_collision_event(self, t, state, *args):
        """Event for when projectile hits the ground"""
        return state[2]  # Returns 0 when z=0 (ground level)

    ground_collision_event.terminal = True
    ground_collision_event.direction = -1

    def simulate_trajectory(self, muzzle_velocity, elevation_angle,
        ↪ azimuth_angle,
                            wind_velocity, initial_altitude, target_distance,
                                ↪ max_time=60):
        """
        Simulate projectile trajectory and find impact point
        """
        # Convert spherical to Cartesian coordinates for initial velocity
        elevation_rad = np.radians(elevation_angle)
        azimuth_rad = np.radians(azimuth_angle)

        vx0 = muzzle_velocity * np.cos(elevation_rad) * np.cos(azimuth_rad)
        vy0 = muzzle_velocity * np.cos(elevation_rad) * np.sin(azimuth_rad)
        vz0 = muzzle_velocity * np.sin(elevation_rad)

        # Initial state [x, y, z, vx, vy, vz]
        initial_state = np.array([0, 0, initial_altitude, vx0, vy0, vz0])

        # Create a local ground event that returns to the launch altitude (
            ↪ handles cases where cannon and target share altitude)
        def _ground_event(t, state, *args):
            # stop when z returns to the initial altitude (descending)
            return state[2] - initial_altitude
```

```
        _ground_event.terminal = True
        _ground_event.direction = -1

        # Solve trajectory
        solution = solve_ivp(
            self.projectile_dynamics,
            [0, max_time],
            initial_state,
            args=(muzzle_velocity, elevation_angle, azimuth_angle,
                ↪ wind_velocity),
            method='RK45',
            dense_output=True,
            events=[_ground_event],
            rtol=1e-8,
            atol=1e-10,
            max_step=0.1
        )

        return solution

    def calculate_impact(self, solution, target_distance):
        """Calculate impact point and error relative to target"""
        if not solution.success or len(solution.t_events[0]) == 0:
            return None, None, float('inf')

        impact_time = solution.t_events[0][0]
        impact_state = solution.sol(impact_time)
        impact_position = impact_state[:3]
        # Compute 2-D Euclidean distance from impact point to the target point
            ↪  (target_distance, 0)
        impact_xy = impact_position[:2]
        target_xy = np.array([target_distance, 0.0])
        error_distance = float(np.linalg.norm(impact_xy - target_xy))

        return impact_position, impact_time, error_distance

    def firing_error(self, parameters, target_distance, initial_altitude,
        ↪ wind_velocity):
        """
        Objective function: minimize distance error to target
        parameters = [muzzle_velocity, elevation_angle, azimuth_angle]
        """
        muzzle_velocity, elevation_angle, azimuth_angle = parameters


        # Check constraints
        if (muzzle_velocity < 100 or muzzle_velocity > self.
            ↪ max_muzzle_velocity or
            elevation_angle < 10 or elevation_angle > 25 or
            azimuth_angle < -180 or azimuth_angle > 180):
            return float('inf')

        # Simulate trajectory
```

```
        solution = self.simulate_trajectory(
            muzzle_velocity, elevation_angle, azimuth_angle,
            wind_velocity, initial_altitude, target_distance
        )

        impact_pos, impact_time, error = self.calculate_impact(solution,
            ↪ target_distance)

        # If no valid impact, return a large penalty
        if impact_pos is None or not np.isfinite(error):
            return 1e6

        # Return only the distance error. Do not penalize flight time here; we
            ↪  are not
        # trying to prefer the fastest solution, only the one that minimizes
            ↪ miss distance.
        return error

    def optimize_firing_solution(self, target_distance, initial_altitude,
        ↪ wind_velocity):
        """
        Find optimal firing parameters using hybrid optimization
        """
        # Parameter bounds for differential_evolution (list of (min,max) pairs
            ↪ )
        de_bounds = [(100, self.max_muzzle_velocity), (1, 25), (-180, 180)]

        # Phase 1: Global optimization
        # Use moderate DE settings to keep runs responsive; increase for
            ↪ production runs
        global_result = differential_evolution(
            self.firing_error,
            de_bounds,
            args=(target_distance, initial_altitude, wind_velocity),
            strategy='best1bin',
            maxiter=20,
            popsize=10,
            tol=0.1,
            seed=42,
            disp=False,
            polish=False
        )

        # Phase 2: Local refinement
        if global_result.success:
            local_result = minimize(
                self.firing_error,
                global_result.x,
                args=(target_distance, initial_altitude, wind_velocity),
                method='SLSQP',
                bounds=[(100, self.max_muzzle_velocity), (1, 25), (-180, 180)
                    ↪ ],
                options={'ftol': 1e-6, 'maxiter': 200}
            )
```

```
            if local_result.success:
                optimal_params = local_result.x
                final_error = local_result.fun
            else:
                optimal_params = global_result.x
                final_error = global_result.fun
        else:
            # Fallback to direct optimization
            optimal_params, final_error = self.direct_optimization(
                target_distance, initial_altitude, wind_velocity
            )

        return optimal_params, final_error

    def direct_optimization(self, target_distance, initial_altitude,
        ↪ wind_velocity):
        """Direct optimization using multiple starting points"""
        best_error = float('inf')
        best_params = None

        # Try different initial guesses
        initial_guesses = [
            [100, 20, 0],
            [200, 20, 0],
            [300, 20, 3],
            [400, 20, -3],   # Medium velocity, medium angle, straight
            [350, 25, 2],   # Upper-bound angle candidate
        ]

        for guess in initial_guesses:
            result = minimize(
                self.firing_error,
                guess,
                args=(target_distance, initial_altitude, wind_velocity),
                method='SLSQP',
                bounds=[(100, self.max_muzzle_velocity), (1, 25), (-180, 180)
                    ↪ ],
                options={'ftol': 1e-6, 'maxiter': 100}
            )

            if result.success and result.fun < best_error:
                best_error = result.fun
                best_params = result.x

        if best_params is None:
            # Default fallback (respect new elevation cap)
            best_params = [400, 20, 0]
            best_error = self.firing_error(best_params, target_distance,
                ↪ initial_altitude, wind_velocity)

        # Run a quick coarse search to discover other feasible solutions (may
            ↪ find lower-velocity or faster-time hits)
        candidates = self.find_candidates_coarse(target_distance,
```

```
        ↪ initial_altitude, wind_velocity,
                                                v_min=100, v_max=self.
                                                    ↪ max_muzzle_velocity,
                                                    ↪ v_step=50,
                                                elev_min=1, elev_max=25,
                                                    ↪ elev_step=2,
                                                azim_choices=[-10, -5, 0, 5,
                                                    ↪ 10],
                                                error_tol=max(5.0, best_error
                                                    ↪  + 1.0))

    if candidates:
        # choose candidate with smallest error, then smallest time
        candidates_sorted = sorted(candidates, key=lambda c: (c[4], c[3]
            ↪ if c[3] is not None else 1e9))
        v_c, elev_c, az_c, t_c, err_c = candidates_sorted[0]
        # Determine current best_time for a fair tie-break
        best_time = None
        if best_params is not None:
            sol_best = self.simulate_trajectory(best_params[0],
                ↪ best_params[1], best_params[2], wind_velocity,
                ↪ initial_altitude, target_distance)
            _, best_time, _ = self.calculate_impact(sol_best,
                ↪ target_distance)

        tol_err = 1e-6
        # Accept candidate if it improves error, or if equal within
            ↪ tolerance and its time is strictly shorter
        if err_c < best_error - tol_err:
            best_params = [v_c, elev_c, az_c]
            best_error = err_c
        elif abs(err_c - best_error) <= tol_err:
            if best_time is None and t_c is not None:
                best_params = [v_c, elev_c, az_c]
                best_error = err_c
            elif (t_c is not None and best_time is not None and t_c <
                ↪ best_time - 1e-6):
                best_params = [v_c, elev_c, az_c]
                best_error = err_c

    return best_params, best_error

def find_candidates_coarse(self, target_distance, initial_altitude,
    ↪ wind_velocity,
                           v_min=100, v_max=None, v_step=50,
                           elev_min=1, elev_max=80, elev_step=2,
                           azim_choices=None, error_tol=20.0):
    """Coarse grid search to find candidate firing solutions quickly.

    Returns list of tuples (muzzle_velocity, elevation, azimuth,
        ↪ impact_time, error)
    """
    if v_max is None:
        v_max = self.max_muzzle_velocity
```

```
        if azim_choices is None:
            # If wind is negligible, only try azimuth = 0 to preserve left-
               ↪ right symmetry
            if np.linalg.norm(wind_velocity) < 1e-1:
                azim_choices = [0.0]
            else:
                azim_choices = [-10.0,-7.5, -5.0,-2.5, 0.0, 2.5, 5.0,7.5,10.0]

        candidates = []
        v_vals = list(range(int(v_min), int(v_max) + 1, int(v_step)))
        elev_vals = list(range(int(elev_min), int(elev_max) + 1, int(elev_step
           ↪ )))

        for v in v_vals:
            for elev in elev_vals:
                for az in azim_choices:
                    sol = self.simulate_trajectory(v, elev, az, wind_velocity,
                       ↪  initial_altitude, target_distance)
                    impact_pos, impact_time, err = self.calculate_impact(sol,
                       ↪ target_distance)
                    if impact_pos is not None and err <= error_tol:
                        candidates.append((v, elev, az, impact_time, err))

        return candidates

    def plot_trajectory_3d(self, muzzle_velocity, elevation_angle,
       ↪ azimuth_angle,
                           wind_velocity, initial_altitude, target_distance):
        """Plot 3D trajectory with wind effects, show lateral Y on side view,
           ↪ and acceleration time series."""
        # simulate
        solution = self.simulate_trajectory(
            muzzle_velocity, elevation_angle, azimuth_angle,
            wind_velocity, initial_altitude, target_distance
        )

        if not solution.success:
            print("Trajectory simulation failed")
            return None, None

        # Determine impact time (use event if available) and generate
           ↪ trajectory points up to impact
        if solution.t_events and len(solution.t_events) > 0 and len(solution.
           ↪ t_events[0]) > 0:
            end_time = float(solution.t_events[0][0])
        else:
            end_time = float(solution.t[-1])

        # avoid zero or negative end_time
        if end_time <= 0:
            end_time = float(solution.t[-1])

        t_eval = np.linspace(0, end_time, 500)
        trajectory = solution.sol(t_eval)
```

```
x_traj = trajectory[0]
y_traj = trajectory[1]
z_traj = trajectory[2]
vx_traj = trajectory[3]
vy_traj = trajectory[4]
vz_traj = trajectory[5]
speed_traj = np.sqrt(vx_traj**2 + vy_traj**2 + vz_traj**2)

# If there's essentially no wind, clean up tiny numerical y and vy
    ↪ drift for display and results
# Use a slightly relaxed tolerance to catch tiny numerical noise from
    ↪ ODE solver
y_tol = 1e-3  # meters
vy_tol = 1e-3  # m/s
if np.linalg.norm(wind_velocity) < 1e-1:
    if np.max(np.abs(y_traj)) < y_tol and np.max(np.abs(vy_traj)) <
        ↪ vy_tol:
        y_traj = np.zeros_like(y_traj)
        vy_traj = np.zeros_like(vy_traj)

# Compute accelerations from aerodynamic drag + gravity at sampled
    ↪ points
acc_traj = np.zeros((3, t_eval.size))
for i in range(t_eval.size):
    vel = np.array([vx_traj[i], vy_traj[i], vz_traj[i]])
    # evaluate drag at the trajectory altitude z_traj[i]
    a_drag = self.wind_force(vel, wind_velocity, altitude=z_traj[i]) /
        ↪  self.mass
    a_gravity = np.array([0.0, 0.0, -self.gravity])
    acc_traj[:, i] = a_drag + a_gravity

acc_x_traj = acc_traj[0]
acc_y_traj = acc_traj[1]
acc_z_traj = acc_traj[2]
acc_mag_traj = np.sqrt(acc_x_traj**2 + acc_y_traj**2 + acc_z_traj**2)

impact_point, impact_time, error = self.calculate_impact(solution,
    ↪ target_distance)
# If no wind and impact lateral displacement is tiny, set to exact
    ↪ zero for results/display
try:
    if impact_point is not None and np.linalg.norm(wind_velocity) < 1e
        ↪ -1 and abs(impact_point[1]) < y_tol:
        impact_point[1] = 0.0
except Exception:
    pass

# Create 2x2 plot grid: 3D trajectory, top-down, side view, and
    ↪ acceleration vs time
fig = plt.figure(figsize=(14, 9))

# 3D trajectory
ax1 = fig.add_subplot(2, 2, 1, projection='3d')
ax1.plot(x_traj, y_traj, z_traj, 'b-', linewidth=2, label='Trajectory'
```

```
    ↪ )
ax1.set_xlabel('X Distance (m)')
ax1.set_ylabel('Y Distance (m)')
ax1.set_zlabel('Altitude (m)')
ax1.set_title('3D Trajectory')
if impact_point is not None:
    ax1.plot([impact_point[0]], [impact_point[1]], [impact_point[2]],
        ↪ 'ro', markersize=8, label='Impact')
ax1.plot([target_distance], [0.0], [initial_altitude], 'rx',
    ↪ markersize=10, label='Target')

# Top-down view (X-Y plane)
ax2 = fig.add_subplot(2, 2, 2)
ax2.plot(x_traj, y_traj, 'g-', linewidth=2, label='Path')
ax2.set_xlabel('X Distance (m)')
ax2.set_ylabel('Y Distance (m)')
ax2.set_title('Top-Down View')
ax2.grid(True, alpha=0.3)
ax2.axis('equal')
pad_xy = max(np.ptp(x_traj), np.ptp(y_traj)) * 0.05 + 1.0
ax2.set_xlim(min(x_traj.min(), 0) - pad_xy, max(x_traj.max(),
    ↪ target_distance) + pad_xy)
ax2.set_ylim(y_traj.min() - pad_xy, y_traj.max() + pad_xy)
if impact_point is not None:
    ax2.plot(impact_point[0], impact_point[1], 'ro', markersize=8,
        ↪ label='Impact')
    target_circle = plt.Circle((target_distance, 0), 10, color='red',
        ↪ fill=False, linestyle='--', label='Target')
    ax2.add_patch(target_circle)

# Side view (X-Z plane) with lateral Y on secondary axis
ax3 = fig.add_subplot(2, 2, 3)
ax3.plot(x_traj, z_traj, 'purple', linewidth=2, label='Altitude')
ax3.set_xlabel('X Distance (m)')
ax3.set_ylabel('Altitude (m)')
ax3.set_title('Side View (with lateral Y)')
ax3.grid(True, alpha=0.3)
if impact_point is not None:
    ax3.plot(impact_point[0], impact_point[2], 'ro', markersize=8,
        ↪ label='Impact')

pad_x = max(np.ptp(x_traj), 1.0) * 0.05 + 1.0
pad_z = max(np.ptp(z_traj), 1.0) * 0.05 + 1.0
ax3.set_xlim(min(x_traj.min(), 0) - pad_x, max(x_traj.max(),
    ↪ target_distance) + pad_x)
ax3.set_ylim(min(z_traj.min(), initial_altitude) - pad_z, max(z_traj.
    ↪ max(), initial_altitude) + pad_z)

try:
    ax3b = ax3.twinx()
    ax3b.plot(x_traj, y_traj, color='tab:green', linestyle='--',
        ↪ linewidth=1.5, label='Lateral Y')
    ax3b.set_ylabel('Lateral Y (m)', color='tab:green')
    ax3b.tick_params(axis='y', labelcolor='tab:green')
```

```
    if impact_point is not None:
        ax3b.plot(impact_point[0], impact_point[1], 'gx', markersize
            ↪ =8)
except Exception:
    pass

# Acceleration vs time plot (bottom-right)
ax4 = fig.add_subplot(2, 2, 4)
ax4.plot(t_eval, acc_mag_traj, color='tab:red', linewidth=2, label='|a
    ↪ |')
ax4.plot(t_eval, acc_x_traj, color='tab:blue', linewidth=1, linestyle=
    ↪ '--', label='ax')
ax4.plot(t_eval, acc_y_traj, color='tab:green', linewidth=1, linestyle
    ↪ ='--', label='ay')
ax4.plot(t_eval, acc_z_traj, color='tab:purple', linewidth=1,
    ↪ linestyle='--', label='az')
ax4.set_xlabel('Time (s)')
ax4.set_ylabel('Acceleration (m/s )')
ax4.set_title('Acceleration vs Time')
ax4.grid(True, alpha=0.3)
ax4.legend()

plt.tight_layout()
try:
    ax1.legend()
    ax2.legend()
    ax3.legend()
except Exception:
    pass
plt.show()

# Save kinematic data (position, velocity, speed, acceleration) to CSV
    ↪  for later analysis (optional)
try:
    csv_path = f"trajectory_v{int(muzzle_velocity)}_e{int(
        ↪ elevation_angle)}_a{int(azimuth_angle)}.csv"
    with open(csv_path, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['time_s', 'x_m', 'y_m', 'z_m',
                         'vx_m_s', 'vy_m_s', 'vz_m_s', 'speed_m_s',
                         'ax_m_s2', 'ay_m_s2', 'az_m_s2', 'a_mag_m_s2'
                             ↪ ])
        for ti, x, y, z, vx, vy, vz, sp, axc, ayc, azc, am in zip(
                t_eval, x_traj, y_traj, z_traj, vx_traj, vy_traj,
                    ↪ vz_traj,
                speed_traj, acc_x_traj, acc_y_traj, acc_z_traj,
                    ↪ acc_mag_traj):
            writer.writerow([
                f"{ti:.6f}", f"{x:.6f}", f"{y:.6f}", f"{z:.6f}",
                f"{vx:.6f}", f"{vy:.6f}", f"{vz:.6f}", f"{sp:.6f}",
                f"{axc:.6f}", f"{ayc:.6f}", f"{azc:.6f}", f"{am:.6f}"
            ])
except Exception:
    pass
```

```
        return impact_point, impact_time

    def analyze_wind_effect(self, target_distance, initial_altitude, base_wind
        ↪ ):
        """Analyze how wind affects the firing solution"""
        wind_magnitudes = [0, 5, 10, 15, 20]  # m/s
        wind_directions = [0, 45, 90, 135, 180]  # degrees

        results = []

        print("WIND EFFECT ANALYSIS")
        print("=" * 60)

        for magnitude in wind_magnitudes:
            for direction in wind_directions:
                # Convert wind direction to velocity vector
                wind_rad = np.radians(direction)
                wind_velocity = np.array([
                    magnitude * np.cos(wind_rad),
                    magnitude * np.sin(wind_rad),
                    0
                ])

                # Find optimal solution (use faster direct optimization here)
                optimal_params, error = self.direct_optimization(
                    target_distance, initial_altitude, wind_velocity
                )

                muzzle_vel, elevation, azimuth = optimal_params

                results.append({
                    'wind_magnitude': magnitude,
                    'wind_direction': direction,
                    'muzzle_velocity': muzzle_vel,
                    'elevation': elevation,
                    'azimuth_correction': azimuth,
                    'error': error
                })

                print(f"Wind: {magnitude:2d} m/s @ {direction:3d}   | "
                      f"V: {muzzle_vel:5.1f} m/s |   : {elevation:4.1f}   | "
                      f"Az: {azimuth:6.1f}   | Error: {error:5.1f} m")

        return results

    def generate_firing_table(self, distances, wind_conditions):
        """Generate firing table for artillery officer"""
        initial_altitude = 500  # Both cannon and target at same altitude

        print("ARTILLERY FIRING TABLE")
        print("=" * 80)
        print("Range (m) | Wind (m/s) | Wind Dir (  ) | Velocity (m/s) |
            ↪ Elevation (  ) | Azimuth (  ) | Error (m)")
```

```
        print("-" * 80)

        firing_table = []

        for distance in distances:
            for wind_mag, wind_dir in wind_conditions:
                # Convert wind to vector
                wind_rad = np.radians(wind_dir)
                wind_velocity = np.array([
                    wind_mag * np.cos(wind_rad),
                    wind_mag * np.sin(wind_rad),
                    0
                ])

                # Calculate solution (use faster direct optimization for table
                    ↪ generation)
                optimal_params, error = self.direct_optimization(
                    distance, initial_altitude, wind_velocity
                )

                muzzle_vel, elevation, azimuth = optimal_params

                firing_table.append({
                    'distance': distance,
                    'wind_magnitude': wind_mag,
                    'wind_direction': wind_dir,
                    'muzzle_velocity': muzzle_vel,
                    'elevation': elevation,
                    'azimuth': azimuth,
                    'error': error
                })

                print(f"{distance:8.0f} | {wind_mag:10.1f} | {wind_dir:12.0f}
                    ↪ | "
                      f"{muzzle_vel:14.1f} | {elevation:12.1f} | {azimuth:10.1
                          ↪ f} | "
                      f"{error:8.1f}")

        return firing_table

def main():
    """Main execution function"""
    # Initialize artillery solver
    artillery = ArtillerySolver3D()

    # Common parameters
    initial_altitude = 500  # Both cannon and target at 500m altitude

    print("3D ARTILLERY FIRING SOLUTION WITH WIND EFFECTS")
    print("=" * 70)

    # Test case 1: 1200m target, no wind
    print("\nCASE 1: 1200m TARGET, NO WIND")
    target_distance = 1200
```

```
wind_velocity = np.array([0, 0, 0])  # No wind

# use faster direct optimization for the initial quick run
optimal_params, error = artillery.direct_optimization(
    target_distance, initial_altitude, wind_velocity
)

muzzle_vel, elevation, azimuth = optimal_params

print(f"Optimal Solution:")
print(f"  Muzzle Velocity: {muzzle_vel:.1f} m/s")
print(f"  Elevation Angle: {elevation:.1f}  ")
print(f"  Azimuth Angle: {azimuth:.1f}  ")
print(f"  Expected Error: {error:.2f} m")

# Plot trajectory
impact_point, impact_time = artillery.plot_trajectory_3d(
    muzzle_vel, elevation, azimuth, wind_velocity, initial_altitude,
        ↪ target_distance
)

if impact_point is not None:
    horizontal_distance = np.linalg.norm(impact_point[:2])
    print(f"Impact Point: ({impact_point[0]:.1f}, {impact_point[1]:.1f}, {
        ↪ impact_point[2]:.1f})")
    print(f"Horizontal Distance: {horizontal_distance:.1f} m")
    print(f"Flight Time: {impact_time:.2f} s")

# Test case 2: 1200m target with crosswind
print("\nCASE 2: 1200m TARGET WITH CROSSWIND (10 m/s from West)")
wind_velocity = np.array([0, 20, 0])  # 10 m/s from West

optimal_params, error = artillery.direct_optimization(
    target_distance, initial_altitude, wind_velocity
)

muzzle_vel, elevation, azimuth = optimal_params

print(f"Optimal Solution:")
print(f"  Muzzle Velocity: {muzzle_vel:.1f} m/s")
print(f"  Elevation Angle: {elevation:.1f}  ")
print(f"  Azimuth Angle: {azimuth:.1f}   (wind compensation)")
print(f"  Expected Error: {error:.2f} m")

# Plot trajectory with wind
impact_point, impact_time = artillery.plot_trajectory_3d(
    muzzle_vel, elevation, azimuth, wind_velocity, initial_altitude,
        ↪ target_distance
)

# # Wind effect analysis
# print("\nCOMPREHENSIVE WIND EFFECT ANALYSIS")
# wind_analysis = artillery.analyze_wind_effect(1200, initial_altitude,
    ↪ 10)
```

```
# Generate firing table for practical use (expanded wind & density sweep)
print("\nPRACTICAL FIRING TABLE (density sweep)")
distances = [1000, 1100, 1200, 1300, 1400, 1500]

# Wind setup: directions every 30 degrees (0..180) and speeds every 2.5 m/
    ↪ s from 0..10
wind_dirs = list(range(0, 181, 30))  # 0,30,...,180
wind_speeds = [round(2.5 * i, 1) for i in range(0, 5)]  #
    ↪ 0.0,2.5,5.0,7.5,10.0
wind_conditions = [(s, d) for d in wind_dirs for s in wind_speeds]

# Air density sweep: 1.1 .. 1.3 kg/m^3 (step 0.1)
densities = [round(1.1 + 0.1 * i, 1) for i in range(0, 3)]  # 1.1,1.2,1.3

# Ensure output directory exists
out_dir = os.path.abspath('.')
print(f"Writing CSVs and plots to: {out_dir}")

# Store per-density summaries for a combined plot
all_density_summaries = {}

for rho in densities:
    artillery.air_density = float(rho)
    print(f"\nRunning firing table for air density = {rho:.1f} kg/m^3;
        ↪ wind cases per distance = {len(wind_conditions)}")

    firing_table = artillery.generate_firing_table(distances,
        ↪ wind_conditions)

    # Save CSV for this density
    csv_path = os.path.join(out_dir, f"firing_table_density_{rho:.1f}.csv"
        ↪ )
    try:
        with open(csv_path, 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(['distance', 'wind_magnitude', 'wind_direction
                ↪ ', 'muzzle_velocity', 'elevation', 'azimuth', 'error'])
            for row in firing_table:
                writer.writerow([
                    row['distance'], row['wind_magnitude'], row['
                        ↪ wind_direction'],
                    row['muzzle_velocity'], row['elevation'], row['azimuth
                        ↪ '], row['error']
                ])
    except Exception as e:
        print(f"Failed to write CSV {csv_path}: {e}")

    # Compute average muzzle velocity per distance for this density
    avg_vs = []
    for distance in distances:
        distance_data = [item for item in firing_table if item['distance']
            ↪  == distance]
        if distance_data:
```

```
                avg_velocity = float(np.mean([item['muzzle_velocity'] for item
                    ↪  in distance_data]))
            else:
                avg_velocity = float('nan')
            avg_vs.append(avg_velocity)

        all_density_summaries[rho] = avg_vs

        # Plot per-density summary
        fig, ax = plt.subplots(figsize=(8, 5))
        ax.plot(distances, avg_vs, marker='o')
        ax.set_title(f"Avg muzzle velocity vs distance (rho={rho:.1f} kg/m^3)"
            ↪ )
        ax.set_xlabel("Distance (m)")
        ax.set_ylabel("Avg muzzle velocity (m/s)")
        ax.grid(True)
        png_path = os.path.join(out_dir, f"avg_velocity_rho_{rho:.1f}.png")
        try:
            fig.savefig(png_path)
            plt.close(fig)
            print(f"Saved {csv_path} and {png_path}")
        except Exception as e:
            print(f"Failed to save plot {png_path}: {e}")

    # Combined plot across densities
    try:
        fig, ax = plt.subplots(figsize=(10, 6))
        for rho in sorted(all_density_summaries.keys()):
            ax.plot(distances, all_density_summaries[rho], marker='o', label=f
                ↪ "rho={rho:.1f}")
        ax.set_title("Avg muzzle velocity vs distance for densities 1.1  1  .3
            ↪ kg/m^3")
        ax.set_xlabel("Distance (m)")
        ax.set_ylabel("Avg muzzle velocity (m/s)")
        ax.grid(True)
        ax.legend()
        combo_png = os.path.join(out_dir, "avg_velocity_all_densities.png")
        fig.savefig(combo_png)
        plt.show()
        print(f"Saved combined plot: {combo_png}")
    except Exception as e:
        print(f"Failed to create combined plot: {e}")

if __name__ == "__main__":
    main()
```

Fit approximate equations:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, HuberRegressor,
    ↪ RANSACRegressor
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score, mean_absolute_error
import os
import glob

class ArtilleryFormulaFitter:
    def __init__(self):
        # Choose robust fitting method: 'huber' (default) or 'ransac'
        self.robust_method = 'huber'
        self.robust_alpha = 0.01
        self.ransac_residual_threshold = None
        # Elevation peak weighting: center (deg) and sigma (deg) for Gaussian
            ↪ weights
        # If center is None, we'll use the median elevation from the training
            ↪ targets.
        self.elevation_peak_center = None
        self.elevation_peak_sigma = 1.5

        self.velocity_model = None
        self.elevation_model = None
        self.azimuth_model = None
        self.density_correction_model = None

    def load_firing_table_data(self, file_pattern="firing_table_density_*.csv"
        ↪ ):
        """Load firing table data for all densities.

        Returns a combined pandas DataFrame with an added 'density' column
        extracted from each filename.
        """
        all_data = []

        #
        files = glob.glob(file_pattern)
        if not files:
            raise FileNotFoundError(f"No files found matching pattern: {
                ↪ file_pattern}")

        print(f"Found {len(files)} data files:")
        for file in files:
            #
            density = float(file.split('_')[-1].replace('.csv', ''))
            print(f"  {file} -> density: {density}")

            df = pd.read_csv(file)
            df['density'] = density  #
            all_data.append(df)
```

```
        #
        combined_df = pd.concat(all_data, ignore_index=True)
        print(f"Loaded {len(combined_df)} total data points")
        return combined_df

    def prepare_features(self, df):
        """Prepare feature matrix, including trig transforms for wind
            ↪ direction."""
        X = df[['distance', 'wind_magnitude', 'wind_direction', 'density']].
            ↪ copy()

        #
        X['wind_dir_sin'] = np.sin(np.radians(X['wind_direction']))
        X['wind_dir_cos'] = np.cos(np.radians(X['wind_direction']))

        #
        X['distance_wind_speed'] = X['distance'] * X['wind_magnitude']
        X['wind_speed_dir_sin'] = X['wind_magnitude'] * X['wind_dir_sin']
        X['wind_speed_dir_cos'] = X['wind_magnitude'] * X['wind_dir_cos']

        #
        X['density_inv_sqrt'] = 1.0 / np.sqrt(X['density'])
        X['distance_density'] = X['distance'] * X['density_inv_sqrt']

        return X

    def fit_velocity_formula(self, X, y_velocity):
        """Fit a simple linear formula for muzzle velocity."""
        #
        velocity_features = ['distance', 'wind_magnitude', 'wind_dir_cos', '
            ↪ density_inv_sqrt']
        X_velocity = X[velocity_features]

        # Choose robust fitting strategy: RANSAC focuses on the largest
            ↪ consensus set
        # (the peak of the distribution); Huber reduces outlier influence
            ↪ while
        # keeping smooth loss near zero residuals.
        if self.robust_method == 'ransac':
            base = LinearRegression()
            model = RANSACRegressor(base_estimator=base, residual_threshold=
                ↪ self.ransac_residual_threshold)
            model.fit(X_velocity, y_velocity)
            coef = model.estimator_.coef_
            intercept = model.estimator_.intercept_
        else:
            model = HuberRegressor(alpha=self.robust_alpha, max_iter=1000)
            model.fit(X_velocity, y_velocity)
            coef = model.coef_
            intercept = model.intercept_

        self.velocity_model = model
```

```
        # compute fit quality
        y_pred = self.velocity_model.predict(X_velocity)
        r2 = r2_score(y_velocity, y_pred)
        mae = mean_absolute_error(y_velocity, y_pred)

        method_name = 'RANSAC' if self.robust_method == 'ransac' else 'Huber'
        print(f"Velocity model  R  : {r2:.4f}, MAE: {mae:.2f} m/s ({method_name
            ↪ })")

        return {
            'coefficients': coef,
            'intercept': intercept,
            'features': velocity_features,
            'r2': r2,
            'mae': mae
        }

    def fit_elevation_formula(self, X, y_elevation):
        """Fit a simple linear formula for elevation angle."""
        #
        elevation_features = ['distance', 'wind_magnitude', 'wind_dir_sin']
        X_elevation = X[elevation_features]

        # If the user wants to focus strictly on a narrow elevation band,
            ↪ filter to that range.
        # New attributes (can be set by caller): elevation_focus_min,
            ↪ elevation_focus_max
        focus_min = getattr(self, 'elevation_focus_min', None)
        focus_max = getattr(self, 'elevation_focus_max', None)

        if focus_min is not None and focus_max is not None:
            # build mask on the target elevations
            mask = (y_elevation >= float(focus_min)) & (y_elevation <= float(
                ↪ focus_max))
            n_in = int(np.sum(mask))
            if n_in < 10:
                print(f"Warning: only {n_in} elevation samples in [{focus_min
                    ↪ },{focus_max}]  , falling back to weighted fit.")
            else:
                # restrict training data to the focus band
                X_elev_fit = X_elevation.loc[mask].reset_index(drop=True)
                y_elev_fit = y_elevation.loc[mask].reset_index(drop=True)

                if self.robust_method == 'ransac':
                    base = LinearRegression()
                    model = RANSACRegressor(base_estimator=base,
                        ↪ residual_threshold=self.ransac_residual_threshold)
                    model.fit(X_elev_fit, y_elev_fit)
                    coef = model.estimator_.coef_
                    intercept = model.estimator_.intercept_
                else:
                    model = HuberRegressor(alpha=self.robust_alpha, max_iter
                        ↪ =1000)
                    model.fit(X_elev_fit, y_elev_fit)
```

```
            coef = model.coef_
            intercept = model.intercept_

        self.elevation_model = model

        y_pred = self.elevation_model.predict(X_elev_fit)
        r2 = r2_score(y_elev_fit, y_pred)
        mae = mean_absolute_error(y_elev_fit, y_pred)

        method_name = 'RANSAC' if self.robust_method == 'ransac' else
            ↪ 'Huber'
        print(f"Elevation model (focused {focus_min}-{focus_max}  )
            ↪ R  : {r2:.4f}, MAE: {mae:.2f}   ({method_name}), n={n_in
            ↪ }")

        return {
            'coefficients': coef,
            'intercept': intercept,
            'features': elevation_features,
            'r2': r2,
            'mae': mae
        }

# If not focusing or not enough samples, fall back to weighted fit
    ↪ over all data
# Build sample weights that emphasize the peak (most common) elevation
    ↪  values.
center = self.elevation_peak_center if self.elevation_peak_center is
    ↪ not None else np.median(y_elevation)
sigma = max(0.1, float(self.elevation_peak_sigma))
# Gaussian weights centered on the peak; points near center get weight
    ↪ ~1, far points downweighted
weights = np.exp(-0.5 * ((y_elevation - center) / sigma) ** 2)
# normalize weights to have mean=1 to keep loss scale similar
weights = weights / np.mean(weights)

if self.robust_method == 'ransac':
    base = LinearRegression()
    model = RANSACRegressor(base_estimator=base, residual_threshold=
        ↪ self.ransac_residual_threshold)
    model.fit(X_elevation, y_elevation, sample_weight=weights)
    coef = model.estimator_.coef_
    intercept = model.estimator_.intercept_
else:
    model = HuberRegressor(alpha=self.robust_alpha, max_iter=1000)
    model.fit(X_elevation, y_elevation, sample_weight=weights)
    coef = model.coef_
    intercept = model.intercept_

self.elevation_model = model

y_pred = self.elevation_model.predict(X_elevation)
r2 = r2_score(y_elevation, y_pred)
mae = mean_absolute_error(y_elevation, y_pred)
```

```
        method_name = 'RANSAC' if self.robust_method == 'ransac' else 'Huber'
        print(f"Elevation model  R  : {r2:.4f}, MAE: {mae:.2f}   ({method_name
            ↪ })")

        return {
            'coefficients': coef,
            'intercept': intercept,
            'features': elevation_features,
            'r2': r2,
            'mae': mae
        }

    def fit_azimuth_formula(self, X, y_azimuth):
        """Fit a simple linear formula for azimuth correction."""
        #
        azimuth_features = ['wind_speed_dir_sin']
        X_azimuth = X[azimuth_features]

        if self.robust_method == 'ransac':
            base = LinearRegression()
            model = RANSACRegressor(base_estimator=base, residual_threshold=
                ↪ self.ransac_residual_threshold)
            model.fit(X_azimuth, y_azimuth)
            coef = model.estimator_.coef_
            intercept = model.estimator_.intercept_
        else:
            model = HuberRegressor(alpha=self.robust_alpha, max_iter=1000)
            model.fit(X_azimuth, y_azimuth)
            coef = model.coef_
            intercept = model.intercept_

        self.azimuth_model = model

        y_pred = self.azimuth_model.predict(X_azimuth)
        r2 = r2_score(y_azimuth, y_pred)
        mae = mean_absolute_error(y_azimuth, y_pred)

        method_name = 'RANSAC' if self.robust_method == 'ransac' else 'Huber'
        print(f"Azimuth model  R  : {r2:.4f}, MAE: {mae:.2f}   ({method_name})"
            ↪ )

        return {
            'coefficients': coef,
            'intercept': intercept,
            'features': azimuth_features,
            'r2': r2,
            'mae': mae
        }

    def fit_all_formulas(self, df):
        """Fit all empirical formulas (velocity, elevation, azimuth)."""
        print("Preparing features...")
        X = self.prepare_features(df)
```

```
        print("\nFitting velocity formula...")
        velocity_result = self.fit_velocity_formula(X, df['muzzle_velocity'])

        print("\nFitting elevation formula...")
        elevation_result = self.fit_elevation_formula(X, df['elevation'])

        print("\nFitting azimuth formula...")
        azimuth_result = self.fit_azimuth_formula(X, df['azimuth'])

        return {
            'velocity': velocity_result,
            'elevation': elevation_result,
            'azimuth': azimuth_result
        }

    def generate_manual_formulas(self, fit_results):
        """Generate human-readable/manual calculation formulas from fit
           ↪ results."""
        v = fit_results['velocity']
        e = fit_results['elevation']
        a = fit_results['azimuth']

        #
        v_coef = v['coefficients']
        v_int = v['intercept']

        e_coef = e['coefficients']
        e_int = e['intercept']

        a_coef = a['coefficients']
        a_int = a['intercept']

        manual_formulas = {
            'velocity': f"v = {v_int:.1f} + {v_coef[0]:.4f}         + {v_coef
               ↪ [1]:.4f}         + {v_coef[2]:.4f} cos (      ) + {v_coef
               ↪ [3]:.4f}/         ",
            'elevation': f"   = {e_int:.1f} + {e_coef[0]:.4f}         + {
               ↪ e_coef[1]:.4f}         + {e_coef[2]:.4f} sin (      )",
            'azimuth': f"   = {a_int:.3f} + {a_coef[0]:.4f}     sin     (
               ↪       )",

            'simplified_velocity': f"v = ({v_int:.0f} + {v_coef[0]:.3f} D  + {
               ↪ v_coef[1]:.3f} W  + {v_coef[2]:.3f}  Wcos  ( _w ))    (1/
               ↪      )^{v_coef[3]:.3f}",
            'simplified_elevation': f"   = {e_int:.1f} + {e_coef[0]:.4f} D  +
               ↪ {e_coef[1]:.3f} W  + {e_coef[2]:.3f}  Wsin  ( _w )",
            'simplified_azimuth': f"   = {a_coef[0]:.3f}    W    sin( _w )",

            'coefficients': {
                'velocity_base': v_int,
                'velocity_distance_coef': v_coef[0],
                'velocity_wind_coef': v_coef[1],
                'velocity_wind_dir_coef': v_coef[2],
```

```
            'velocity_density_coef': v_coef[3],

            'elevation_base': e_int,
            'elevation_distance_coef': e_coef[0],
            'elevation_wind_coef': e_coef[1],
            'elevation_wind_dir_coef': e_coef[2],

            'azimuth_base': a_int,
            'azimuth_wind_coef': a_coef[0]
        }
    }

    return manual_formulas

def calculate_with_formulas(self, distance, wind_speed, wind_dir, density
    ↪ =1.2):
    """Calculate velocity/elevation/azimuth using the fitted formulas."""
    if not all([self.velocity_model, self.elevation_model, self.
        ↪ azimuth_model]):
        raise ValueError("Models not fitted yet. Call fit_all_formulas
            ↪ first.")

    #
    wind_dir_rad = np.radians(wind_dir)
    features_dict = {
        'distance': distance,
        'wind_magnitude': wind_speed,
        'wind_direction': wind_dir,
        'density': density,
        'wind_dir_sin': np.sin(wind_dir_rad),
        'wind_dir_cos': np.cos(wind_dir_rad),
        'wind_speed_dir_sin': wind_speed * np.sin(wind_dir_rad),
        'density_inv_sqrt': 1.0 / np.sqrt(density)
    }

    #              - pass a DataFrame with the original feature names to
        ↪ avoid sklearn warnings
    velocity_features = ['distance', 'wind_magnitude', 'wind_dir_cos', '
        ↪ density_inv_sqrt']
    X_velocity = pd.DataFrame([{f: features_dict[f] for f in
        ↪ velocity_features}])
    velocity = self.velocity_model.predict(X_velocity)[0]

    #
    elevation_features = ['distance', 'wind_magnitude', 'wind_dir_sin']
    X_elevation = pd.DataFrame([{f: features_dict[f] for f in
        ↪ elevation_features}])
    elevation = self.elevation_model.predict(X_elevation)[0]

    #
    azimuth_features = ['wind_speed_dir_sin']
    X_azimuth = pd.DataFrame([{f: features_dict[f] for f in
        ↪ azimuth_features}])
    azimuth = self.azimuth_model.predict(X_azimuth)[0]
```

```
        return {
            'velocity': max(100, min(450, velocity)),  #
            'elevation': max(10, min(25, elevation)),  #      10     -25
            'azimuth': azimuth
        }

    def plot_fitting_results(self, df, fit_results):
        """Plot fitting diagnostics as two separate figures:
        - Figure 1: predicted vs actual scatter plots for velocity, elevation,
            ↪  azimuth
        - Figure 2: error histograms for each quantity
        Returns the same error statistics dict as before.
        """
        X = self.prepare_features(df)

        # Create and save six separate JPGs (three scatter plots and three
            ↪ histograms)
        # Predictions
        y_velocity_pred = self.velocity_model.predict(X[fit_results['velocity'
            ↪ ]['features']])
        elev_features = fit_results['elevation']['features']
        y_elevation_pred_full = self.elevation_model.predict(X[elev_features])
        focus_min = getattr(self, 'elevation_focus_min', None)
        focus_max = getattr(self, 'elevation_focus_max', None)
        y_azimuth_pred = self.azimuth_model.predict(X[fit_results['azimuth']['
            ↪ features']])

        # --- Scatter: Velocity ---
        fig_v, ax_v = plt.subplots(figsize=(6, 5))
        ax_v.scatter(df['muzzle_velocity'], y_velocity_pred, alpha=0.6)
        ax_v.plot([df['muzzle_velocity'].min(), df['muzzle_velocity'].max()],
                  [df['muzzle_velocity'].min(), df['muzzle_velocity'].max()],
                     ↪ 'r--')
        ax_v.set_xlabel('Actual velocity (m/s)')
        ax_v.set_ylabel('Predicted velocity (m/s)')
        ax_v.set_title(f'Velocity fit (R ={fit_results["velocity"]["r2"]:.3f
            ↪ })')
        ax_v.grid(True, alpha=0.3)
        fig_v.tight_layout()
        fig_v.savefig('fitting_velocity_scatter.jpg', dpi=200)
        plt.close(fig_v)

        # --- Scatter: Elevation (focused if configured) ---
        fig_e, ax_e = plt.subplots(figsize=(6, 5))
        if focus_min is not None and focus_max is not None:
            mask = (df['elevation'] >= float(focus_min)) & (df['elevation'] <=
                ↪  float(focus_max))
            n_mask = int(mask.sum())
            if n_mask > 0:
                y_elev_series = pd.Series(y_elevation_pred_full, index=df.
                    ↪ index)
                ax_e.scatter(df.loc[mask, 'elevation'], y_elev_series.loc[mask
                    ↪ ], alpha=0.7)
```

```
        ax_e.plot([focus_min, focus_max], [focus_min, focus_max], 'r--
            ↪ ')
        ax_e.set_title(f'Elevation fit (focused {focus_min}-{focus_max
            ↪ }  , n={n_mask})')
    else:
        ax_e.scatter(df['elevation'], y_elevation_pred_full, alpha
            ↪ =0.6)
        ax_e.plot([df['elevation'].min(), df['elevation'].max()],
                  [df['elevation'].min(), df['elevation'].max()], 'r--
                      ↪ ')
        ax_e.set_title(f'Elevation fit (R ={fit_results["elevation
            ↪ "]["r2"]:.3f})')
else:
    ax_e.scatter(df['elevation'], y_elevation_pred_full, alpha=0.6)
    ax_e.plot([df['elevation'].min(), df['elevation'].max()],
              [df['elevation'].min(), df['elevation'].max()], 'r--')
    ax_e.set_title(f'Elevation fit (R ={fit_results["elevation"]["r2
        ↪ "]:.3f})')

ax_e.set_xlabel('Actual elevation (  )')
ax_e.set_ylabel('Predicted elevation (  )')
ax_e.grid(True, alpha=0.3)
fig_e.tight_layout()
fig_e.savefig('fitting_elevation_scatter.jpg', dpi=200)
plt.close(fig_e)

# --- Scatter: Azimuth ---
fig_a, ax_a = plt.subplots(figsize=(6, 5))
ax_a.scatter(df['azimuth'], y_azimuth_pred, alpha=0.6)
ax_a.plot([df['azimuth'].min(), df['azimuth'].max()],
          [df['azimuth'].min(), df['azimuth'].max()], 'r--')
ax_a.set_xlabel('Actual azimuth (  )')
ax_a.set_ylabel('Predicted azimuth (  )')
ax_a.set_title(f'Azimuth fit (R ={fit_results["azimuth"]["r2"]:.3f})'
    ↪ )
ax_a.grid(True, alpha=0.3)
fig_a.tight_layout()
fig_a.savefig('fitting_azimuth_scatter.jpg', dpi=200)
plt.close(fig_a)

# --- Histogram: Velocity error ---
velocity_error = y_velocity_pred - df['muzzle_velocity']
fig_vh, ax_vh = plt.subplots(figsize=(6, 5))
ax_vh.hist(velocity_error, bins=30, alpha=0.7)
ax_vh.axvline(velocity_error.mean(), color='red', linestyle='--',
              label=f'Mean error: {velocity_error.mean():.2f} m/s')
ax_vh.set_xlabel('Velocity error (m/s)')
ax_vh.set_ylabel('Count')
ax_vh.set_title('Velocity error distribution')
ax_vh.legend()
ax_vh.grid(True, alpha=0.3)
fig_vh.tight_layout()
fig_vh.savefig('fitting_velocity_error.jpg', dpi=200)
plt.close(fig_vh)
```

```
# --- Histogram: Elevation error (focused if configured) ---
y_elev_series = pd.Series(y_elevation_pred_full, index=df.index)
fig_eh, ax_eh = plt.subplots(figsize=(6, 5))
if focus_min is not None and focus_max is not None and (df['elevation'
    ↪ ] >= float(focus_min)).any():
    mask = (df['elevation'] >= float(focus_min)) & (df['elevation'] <=
        ↪  float(focus_max))
    elevation_error = y_elev_series.loc[mask] - df.loc[mask, '
        ↪ elevation']
    ax_eh.hist(elevation_error, bins=30, alpha=0.7)
    ax_eh.axvline(elevation_error.mean(), color='red', linestyle='--',
                  label=f'Mean error (focus): {elevation_error.mean()
                      ↪ :.2f}  ')
    ax_eh.set_title(f'Elevation error (focused {focus_min}-{focus_max}
        ↪   , n={int(mask.sum())})')
else:
    elevation_error = y_elev_series - df['elevation']
    ax_eh.hist(elevation_error, bins=30, alpha=0.7)
    ax_eh.axvline(elevation_error.mean(), color='red', linestyle='--',
                  label=f'Mean error: {elevation_error.mean():.2f}  '
                      ↪ )
ax_eh.set_xlabel('Elevation error (  )')
ax_eh.set_ylabel('Count')
ax_eh.set_title('Elevation error distribution')
ax_eh.legend()
ax_eh.grid(True, alpha=0.3)
fig_eh.tight_layout()
fig_eh.savefig('fitting_elevation_error.jpg', dpi=200)
plt.close(fig_eh)

# --- Histogram: Azimuth error ---
azimuth_error = y_azimuth_pred - df['azimuth']
fig_ah, ax_ah = plt.subplots(figsize=(6, 5))
ax_ah.hist(azimuth_error, bins=30, alpha=0.7)
ax_ah.axvline(azimuth_error.mean(), color='red', linestyle='--',
              label=f'Mean error: {azimuth_error.mean():.2f}  ')
ax_ah.set_xlabel('Azimuth error (  )')
ax_ah.set_ylabel('Count')
ax_ah.set_title('Azimuth error distribution')
ax_ah.legend()
ax_ah.grid(True, alpha=0.3)
fig_ah.tight_layout()
fig_ah.savefig('fitting_azimuth_error.jpg', dpi=200)
plt.close(fig_ah)

return {
    'velocity_error_stats': {
        'mean': velocity_error.mean(),
        'std': velocity_error.std(),
        'max': float(np.max(np.abs(velocity_error)))
    },
    'elevation_error_stats': {
        'mean': elevation_error.mean(),
```

```
                'std': elevation_error.std(),
                'max': float(np.max(np.abs(elevation_error)))
            },
            'azimuth_error_stats': {
                'mean': azimuth_error.mean(),
                'std': azimuth_error.std(),
                'max': float(np.max(np.abs(azimuth_error)))
            }
        }

def main():
    """Main: fit empirical formulas from firing table CSVs."""
    fitter = ArtilleryFormulaFitter()

    try:
        # Load data
        print("Loading firing table data...")
        df = fitter.load_firing_table_data()

        # Fit formulas
        print("\nFitting empirical formulas...")

        # --- IMPORTANT: by default focus elevation fitting on 23-25 degrees
            ↪ ---
        # This enforces the requested behavior: train elevation model only on
        # samples whose elevation lies in [23, 25]. You can change or disable
        # this by editing these attributes before running.
        fitter.elevation_focus_min = 23.0
        fitter.elevation_focus_max = 25.0
        fitter.elevation_peak_sigma = 1.0
        # Keep Huber as default robust method; switch to 'ransac' for
            ↪ consensus-only
        fitter.robust_method = getattr(fitter, 'robust_method', 'huber')

        print(f"Focusing elevation fit on range [{fitter.elevation_focus_min},
            ↪  {fitter.elevation_focus_max}]  ")
        fit_results = fitter.fit_all_formulas(df)

        # Generate manual formulas
        print("\nGenerating manual calculation formulas...")
        formulas = fitter.generate_manual_formulas(fit_results)

        # Display fitted formulas
        print("\n" + "="*60)
        print("FITTED ARTILLERY FORMULAS")
        print("="*60)

        print("\nVELOCITY FORMULA:")
        print(formulas['velocity'])
        print(f"Simplified: {formulas['simplified_velocity']}")

        print("\nELEVATION FORMULA:")
        print(formulas['elevation'])
        print(f"Simplified: {formulas['simplified_elevation']}")
```

```
print("\nAZIMUTH FORMULA:")
print(formulas['azimuth'])
print(f"Simplified: {formulas['simplified_azimuth']}")

# Show numerical coefficients
print("\nNUMERICAL COEFFICIENTS:")
coeffs = formulas['coefficients']
print(f"Velocity: base={coeffs['velocity_base']:.2f}, distance_coef={
    ↪ coeffs['velocity_distance_coef']:.4f}")
print(f"          wind_coef={coeffs['velocity_wind_coef']:.4f},
    ↪ wind_dir_coef={coeffs['velocity_wind_dir_coef']:.4f}")
print(f"          density_coef={coeffs['velocity_density_coef']:.4f}")

print(f"Elevation: base={coeffs['elevation_base']:.2f}, distance_coef
    ↪ ={coeffs['elevation_distance_coef']:.4f}")
print(f"           wind_coef={coeffs['elevation_wind_coef']:.4f},
    ↪ wind_dir_coef={coeffs['elevation_wind_dir_coef']:.4f}")

print(f"Azimuth: base={coeffs['azimuth_base']:.4f}, wind_coef={coeffs
    ↪ ['azimuth_wind_coef']:.4f}")

# Validate formulas with sample cases
print("\n" + "="*60)
print("FORMULA VALIDATION")
print("="*60)

test_cases = [
    (1200, 10, 90, 1.2),
    (1500, 5, 0, 1.2),
    (1000, 15, 180, 1.1),
    (1300, 8, 270, 1.3)
]

print("\nTest calculations:")
print("Distance(m) | Wind speed(m/s) | Wind dir(  ) | Density |
    ↪ Velocity(m/s) | Elevation(  ) | Azimuth(  )")
print("-" * 80)

for distance, wind_speed, wind_dir, density in test_cases:
    result = fitter.calculate_with_formulas(distance, wind_speed,
        ↪ wind_dir, density)
    print(f"{distance:7.0f} | {wind_speed:9.1f} | {wind_dir:8.0f} | {
        ↪ density:4.1f} | "
          f"{result['velocity']:9.1f} | {result['elevation']:8.1f} | {
              ↪ result['azimuth']:9.1f}")

# Plot fitting results
print("\nPlotting fitting results...")
error_stats = fitter.plot_fitting_results(df, fit_results)

print("\nError Statistics:")
print(f"Velocity: mean={error_stats['velocity_error_stats']['mean']:.2
    ↪ f} m/s, "
```

```
            f"std={error_stats['velocity_error_stats']['std']:.2f} m/s")
        print(f"Elevation: mean={error_stats['elevation_error_stats']['mean
            ↪ ']:.2f}  , "
            f"std={error_stats['elevation_error_stats']['std']:.2f}  ")
        print(f"Azimuth: mean={error_stats['azimuth_error_stats']['mean']:.2f}
            ↪   , "
            f"std={error_stats['azimuth_error_stats']['std']:.2f}  ")

        # Produce final manual calculation guide (English)
        print("\n" + "="*60)
        print("FINAL MANUAL CALCULATION GUIDE")
        print("="*60)

        manual_guide = f"""
Empirical artillery manual (derived from fitted data)
==============================================

1. Velocity formula:
   v = {coeffs['velocity_base']:.1f} + {coeffs['velocity_distance_coef']:.4f}
      ↪  D  + {coeffs['velocity_wind_coef']:.4f} W  + {coeffs['
      ↪ velocity_wind_dir_coef']:.4f}  Wcos  (phi) + {coeffs['
      ↪ velocity_density_coef']:.4f}/

2. Elevation formula:
      = {coeffs['elevation_base']:.1f} + {coeffs['elevation_distance_coef']:.4
      ↪ f} D  + {coeffs['elevation_wind_coef']:.4f} W  + {coeffs['
      ↪ elevation_wind_dir_coef']:.4f}  Wsin  (phi)

3. Azimuth formula:
      = {coeffs['azimuth_wind_coef']:.4f}    W    sin(phi)

Where:
- D: target distance (m), typical range: 1000-1500 m
- W: wind speed (m/s)
- phi: wind direction (degrees), convention used in features
-   : air density (kg/m  ), used range: 1.1-1.3

Quick reference:
- Per +100 m distance, velocity changes     {coeffs['velocity_distance_coef
   ↪ ']*100:.1f} m/s
- Per +1 m/s headwind, velocity changes     {coeffs['velocity_wind_coef'] +
   ↪ coeffs['velocity_wind_dir_coef']:.3f} m/s
- Density change 1.2  1  .1 causes velocity change     {(coeffs['
   ↪ velocity_density_coef']*(1/np.sqrt(1.1)-1/np.sqrt(1.2))):.1f} m/s
        """
        print(manual_guide)

    except Exception as e:
        print(f"Error: {e}")
        print("Make sure the CSV files are in the current directory.")

if __name__ == "__main__":
    main()
```